

Contents

1	SGF File Format FF[4]	4
2	Contents	6
3	1. SGF basics	7
4	2. Basic (EBNF) Definition	8
4.1	2.1. EBNF Definition	8
4.2	2.2. Some remarks about properties	9
4.2.1	2.2.1. Property Types (currently five):	9
4.2.2	2.2.2. Property attributes (currently only one)	10
4.2.3	2.2.3. How to handle unknown/faulty properties	10
4.2.4	2.2.4. Private Properties	10
5	3. Property Value Types	12
5.1	3.1. Double	12
5.2	3.2. Text	12
5.2.1	3.2.1. Example:	13
5.3	3.3. SimpleText	13
5.4	3.4. Stone	14
5.5	3.5. Move / Point	14
5.5.1	3.5.1. Compressed point lists	15
6	Contents	16
7	SGF Properties (FF[4])	17
7.1	Move properties	17
7.2	Setup properties	18
7.2.1	Restrictions	18
7.3	Node annotation properties	19
7.4	Move annotation properties	21
7.4.1	Restrictions	21
7.5	Markup properties	21
7.5.1	Restrictions	21
7.6	Root properties	23
7.7	Game info properties	26
7.8	Timing properties	30

7.9	Miscellaneous properties	31
8	Variations	33
8.1	Common pitfalls:	35
9	Go (GM[1])	36
9.1	Move, Point & Stone type	36
9.1.1	How to execute a move	36
9.2	Properties	37
10	FF[4] property index	38
11	FF[4] property index	40
11.1	General properties	40
11.2	Go (GM[1]) specific properties	41
11.3	Lines of Action (GM[9]) specific properties	42
12	Property index of FF[1]-FF[4]	43
13	Dimmed Stones and View	46
13.1	Picture of node 0	46
13.2	Picture of node 1	46
14	SGF FF[4] History	48
14.0.1	2021-12-01	48
14.0.2	2007 - 2016	48
14.0.3	2006-08-06	48
14.0.4	2006-06-25	48
14.0.5	2006-05-13	48
14.0.6	2002-2005	49
14.0.7	2001-08-12	49
14.0.8	2001-01-21	49
14.0.9	2000-10-18	49
14.0.10	2000-01-15	49
14.0.11	2000-01-04	50
14.0.12	1999-12-17	50
14.0.13	1999-01-07	50
15	From FF[3] to FF[4]	51
15.1	General	51
15.2	Properties no longer part of FF[4]	51
15.3	New properties	52
15.4	Changed properties	52
16	Compatibility of FF[4]	53
16.1	General & Types	53
16.2	Properties	54
17	Converting old SGF files to FF[4]	55

17.1	Converting properties	55
17.2	Other conversion issues	56
17.2.1	Property identifiers	56
17.2.2	Property types move & setup	56
17.2.3	Illegal & erroneous files	56
18	SGF User Guide	58
19	1. What is SGF?	59
19.1	Versions	59
19.2	What does an SGF file look like?	59
19.3	SGF applications	60
20	2. General Concepts	61
21	3. ‘Make a Move’ vs. ‘Place a Stone’	62
21.1	Restrictions	62
22	4. Style	63
23	5. Variations	64
24	6. Board markup	66
25	7. Comments and annotations	67
26	8. Game Information	69
26.1	Why should you use game information?	69
26.2	List of game information properties	69
27	1. General SGF FF[4] Example File	73
27.1	Purpose	73
27.2	General comments	73
27.3	Download example SGF file	73
27.4	Gametree 1: properties	73
27.4.1	Variation A: (Moves, comments, annotations)	73
27.4.2	Variation B: (Setup)	74
27.4.3	Variation C: (Markup)	74
27.4.4	Variation D: (Style & text type)	74
27.4.5	Variation E: (Time limits, captures & move numbers)	75
27.5	Gametree 2: Game-info	75
28	2. Print Example Files	76
29	SGF Example Files	77

Chapter 1

SGF File Format FF[4]

This is the official specification of the SGF FF[4] standard.

SGF is the abbreviation of ‘Smart Game Format’. The file format is designed to store game records of board games for two players. It’s a text only, tree based format. Therefore games stored in this format can easily be emailed, posted or processed with text-based tools.

The main purposes of SGF are to store records of played games and to provide features for storing annotated and analyzed games (e.g. board markup, variations).

Last updated: 2021-12-01

See history for changes.

For users

Users guide

An userorientated introduction to SGF files

Tools and more

SGFC

SGF Syntax Checker & Converter

Example file

An example SGF file plus pictures to show the basic and sophisticated features of SGF

For developers

SGF Specification

Basics

- Basic definition and types- Additional examples for variations. - Game specific definitions and types - Go (GM[1]) - Backgammon (GM[6]) - Lines of Action (GM[9]) - Hex (GM[11]) - Amazons (GM[18]) - Octi (GM[19]) - Gess (GM[20]) - Twixt (GM[21])

Properties

- General properties- Game specific properties: - Go (GM[1]) - Backgammon (GM[6]) - Lines of Action (GM[9]) - Hex (GM[11]) - Short index of all properties (alphabetical) - Short index of all properties (sorted by property type & game) Specification Supplements

For users	
UTI-Specification	For Apple systems: a proposal for a Uniform Type Identifier (UTI) for SGF files (draft)
Changes to FF[3]	To get a quick overview of what's new in FF[4] have a look at the changes from FF[3].
Compatibility	Compatibility issues
Converting	Converting old files to FF[4]
Index of properties	Index of all FF[1]-FF[4] properties (alphabetical)
Translations	There were translations in the past, currently all links are broken. Following (outdated) copies are available from archive.org: - Chinese version by Fang Yuan.- Russian version by Pavel Strybuk.- Japanese version by Teruo Namatame.
	Old specifications
FF[1]	Specification of FF[1] by Anders Kierulf
FF[3]	Specification of FF[3] by Martin Müller
Style guide	Style guide by Martin Müller (old but still valid)
	Obsolete or outdated information
List archive	Archive of the original email list from 1996 to 1997
FF[5]	Ideas for FF5 from around 2003 (obsolete)
XGF	proposal for an XML format to replace SGF from around 2003 (obsolete)

Note: Many Go (WeiQi) terms are used throughout the specification, e.g. point is used instead of field or square.

Note: Please pay attention to the difference of *mandatory* (has to be, must not, ...) and *recommended* (suggested, should have, shouldn't ...). Mandatory topics HAVE TO be done exactly this way, otherwise it's illegal. Recommended topics should but don't have to be followed. If the application doesn't obey those suggestions then the 'only' consequence is bad style.

Arno Hollosiahhollosi@xmp.net

Hosted at  Red BeanVirtual solutions for virtual people

Chapter 2

Contents

- SGF basics
 - Basic (EBNF) definition
 - EBNF definition
 - Some remarks about properties
 - * Property types
 - * Property attributes
 - * How to handle unknown/faulty properties
 - * Private Properties
 - Property Value Types
 - Double
 - Text
 - SimpleText
 - Stone
 - Move / Point
 - * Compressed point lists
-

Chapter 3

1. SGF basics

SGF is a text-only format (not a binary format).

It contains game trees, with all their nodes and properties, and nothing more. Thus the file format reflects the regular internal structure of a tree of property lists. There are no exceptions; if a game needs to store some information on file with the document, a (game-specific) property must be defined for that purpose.

Example for tree structure	Tree as seen by the user. The first line is the main line of play, the other lines are variations.
----------------------------	--

There are more examples available.

Node numbering:

When numbering nodes starting with zero is suggested. Nodes should be numbered in the way they are stored in the file.

Example (of file above): root=0, a=1, b=2, c=3, d=4, e=5, f=6, g=7, h=8, i=9 and j=10.

SGF uses the US ASCII char-set for all its property identifiers and property values, except SimpleText & Text. For SimpleText & Text the charset is defined using the CA property.

Chapter 4

2. Basic (EBNF) Definition

The conventions of EBNF are discussed in literature, and on WWW in the Computing Dictionary.

A quick summary:

```
"..." : terminal symbols
[...]  : option: occurs at most once
{...}  : repetition: any number of times, including zero
(...)  : grouping
|      : exclusive or
italics: parameter explained at some other place
```

4.1 2.1. EBNF Definition

```
Collection = GameTree { GameTree }
GameTree   = "(" Sequence { GameTree } ")"
Sequence   = Node { Node }
Node       = ";" { Property }
Property   = PropIdent PropValue { PropValue }
PropIdent  = UcLetter { UcLetter }
PropValue  = "[" CValueType "]"
CValueType = (ValueType | Compose)
ValueType  = (None | Number | Real | Double | Color | SimpleText |
              Text | Point | Move | Stone)
```

White space (space, tab, carriage return, line feed, vertical tab and so on) may appear anywhere between PropValues, Properties, Nodes, Sequences and GameTrees.

There are two types of property lists: ‘list of’ and ‘elist of’.

```
'list of':   PropValue { PropValue }
'elist of':  ((PropValue { PropValue }) | None)
              In other words elist is list or "[]".
```


4.2 2.2. Some remarks about properties

Property-identifiers are defined as keywords using only uppercase letters. Currently there are no more than two uppercase letters per identifier.

The order of properties in a node is not fixed. It may change every time the file is saved and may vary from application to application. Furthermore applications should **not** rely on the order of property values. The order of values might change as well.

Everybody is free to define additional, private properties, as long as they do not interfere with the standard properties defined in this document.

Therefore, if one is writing a SGF reader, it is important to skip unknown properties. An application should issue a warning message when skipping unknown or faulty properties.

Only one of each property is allowed per node, e.g. one cannot have two comments in one node:

```
... ; C[comment1] B [dg] C[comment2] ; ...
```

This is an error.

Each property has a property type. Property types place restrictions on certain properties e.g. in which nodes they are allowed and with which properties they may be combined.

4.2.1 2.2.1. Property Types (currently five):

move Properties of this type concentrate on the move made, not on the position arrived at by this move.
Move properties must not be mixed with setup properties within the same node.
Note: it's bad style to have move properties in root nodes.
(it isn't forbidden though)

setup Properties of this type concentrate on the current position.
Setup properties must not be mixed with move properties within a node.

root Root properties may only appear in root nodes. Root nodes are the first nodes of gametrees, which are direct descendants from a collection (i.e. not gametrees within other gametrees).
They define some global 'attributes' such as board-size, kind of game, used file format etc.

game-info
Game-info properties provide some information about the game played (e.g. who, where, when, what, result, rules, etc.).
These properties are usually stored in root nodes.
When merging a set of games into a single gametree, game infos are stored at the nodes where a game first becomes distinguishable from all other games in the tree.

A node containing game-info properties is called a game-info node.

There may be only one game-info node on any path within the tree, i.e. if some game-info properties occur in one node there may not be any further game-info properties in following nodes:

- a) on the path from the root node to this node.
- b) in the subtree below this node.

- no type. These properties have no special types and may appear anywhere in a collection.

Because of the strict distinction between move and setup properties nodes could also be divided into two classes: move-nodes and setup-nodes. This is important for databases, converting to/from ISHI-format and for some special applications.

4.2.2 2.2.2. Property attributes (currently only one)

inherit

Properties having this attribute affect not only the node containing these properties but also ALL subsequent child nodes as well until a new setting is encountered or the setting gets cleared.

I.e. once set all children (of this node) inherit the values of the 'inherit' type properties.

E.g. VW restricts the view not only of the current node, but of all successors nodes as well. Thus a VW at the beginning of a variation is valid for the whole variation tree.

Inheritance stops, if either a new property is encountered and those values are inherited from now on, or the property value gets cleared, typically by an empty value, e.g. VW[].

4.2.3 2.2.3. How to handle unknown/faulty properties

- Unknown properties and their values should be preserved.

If an application isn't able to preserve unknown properties, then it has to display a warning message.

- Illegally formatted game-info properties should be corrected if possible, otherwise preserved.
- Other illegally formatted properties should be corrected if possible, otherwise deleted.

An application has to display a warning message, if it deletes illegally formatted properties.

4.2.4 2.2.4. Private Properties

Applications may define their own private properties. Some restrictions apply.

Property identifier: private properties **must not** use an identifier used by one of the standard properties. You have to use a new identifier instead. The identifier should consist of up to two uppercase letters. SGF doesn't require to limit the identifier to two letters, but some applications could break otherwise.

Property value: private properties may use one of the value types defined in this document or define their own value type. When using a private value type the application has to escape **every**

"]" with a leading "\". Otherwise the file would become unparseable. Should the value type be a combination of two simpler types then it's suggested that your application uses the **Compose** type.

Chapter 5

3. Property Value Types

```
UcLetter    = "A".."Z"
Digit       = "0".."9"
None        = ""

Number      = [("+","-")] Digit { Digit }
Real        = Number [ "." Digit { Digit } ]

Double      = ("1" | "2")
Color       = ("B" | "W")

SimpleText  = { any character (handling see below) }
Text        = { any character (handling see below) }

Point       = game-specific
Move        = game-specific
Stone       = game-specific

Compose     = ValueType ":" ValueType
```

5.1 3.1. Double

Double values are used for annotation properties. They are called Double because the value is either simple or emphasized. A value of ‘1’ means ‘normal’; ‘2’ means that it is emphasized.

Example:

GB[1] could be displayed as: Good for black

GB[2] could be displayed as: Very good for black

5.2 3.2. Text

Text is a formatted text. White spaces other than linebreaks are converted to space (e.g. no tab, vertical tab, ..).

Formatting:

Soft line break: linebreaks preceded by a "\" (soft linebreaks are converted to "", i.e. they are removed)

Hard line breaks: any other linebreaks encountered

Attention: a single linebreak is represented differently on different systems, e.g. "LF" for DOS, "LF" on Unix. An application should be able to deal with following linebreaks: LF, CR, LF, CRLF.

Applications must be able to handle Texts of any size. The text should be displayed the way it is, though long lines may be word-wrapped, if they don't fit the display.

Escaping: "\" is the escape character. Any char following "\" is inserted verbatim (exception: whitespaces still have to be converted to space!). Following chars have to be escaped, when used in Text: "]", "\", and ":" (only if used in compose data type).

Encoding: texts can be encoded in different charsets. See CA property.

5.2.1 3.2.1. Example:

```
C[Meijin NR: yeah, k4 is won\
derful\
sweat NR: thank you! :)\
dada NR: yup. I like this move too. It's a move only to be expected from a pro. I really like :
jansteen 4d: Can anyone\
  explain [me\] k4?]
```

could be rendered as:

```
Meijin NR: yeah, k4 is wonderful
sweat NR: thank you! :)
dada NR: yup. I like this move too. It's a move only to be expected
from a pro. I really like it :)
jansteen 4d: Can anyone explain [me] k4?
```

5.3 3.3. SimpleText

SimpleText is a simple string. Whitespaces other than space must be converted to space, i.e. there's no newline! Applications must be able to handle SimpleTexts of any size.

Formatting: linebreaks preceded by a "\" are converted to "", i.e. they are removed (same as Text type). All other linebreaks are converted to space (no newline on display!!).

Escaping (same as Text type): "\" is the escape character. Any char following "\" is inserted verbatim (exception: whitespaces still have to be converted to space!). Following chars have to be escaped, when used in SimpleText: "]", "\", and ":" (only if used in compose data type).

Encoding (same as Text type): SimpleTexts can be encoded in different charsets. See CA property.

5.4 3.4. Stone

This type is used to specify the point and the piece that should be placed at that point. If a game doesn't have a distinguishable set of pieces (figures) like e.g. Go (GM[1]) the Stone type is reduced to the Point type below, e.g. "list of stone" becomes "list of point" for that game.

Note: if a property allows "list of stone" a reduction to "list of point" allows compressed point lists.

- *Go, Othello, Gomoku, Renju*: Stone becomes Point
- *Chess*
- *Nine Men's Morris*
- *Chinese chess*
- *Shogi*
- *Backgammon, Lines of Action, Hex, Gess*: Stone becomes Point
- *Amazons*
- *Octi*

5.5 3.5. Move / Point

These two types are game specific.

- Go
- Othello
- Chess
- Gomoku, Renju
- Nine Men's Morris
- Backgammon
- Chinese chess
- Shogi
- Lines of Action
- Hex
- Amazons
- Gess
- Octi

5.5.1 3.5.1. Compressed point lists

The PropValue type “*list of point*” or “*elist of point*” may be compressed.

Compressing is done by specifying rectangles instead of listing every single point in the rectangle. Rectangles are specified by using the upper left and lower right corner of the rectangle.

Definition:

List of point: list of (point | composition of point ":" point)

For the composed type the first point specifies the upper left corner, the second point the lower right corner of the rectangle.

1x1 Rectangles are illegal - they've to be listed as single point.

The definition of ‘point list’ allows both single point [xy] and rectangle [ul:lr] specifiers in any order and combination. However the points have to be unique, i.e. overlap and duplication are forbidden.

To get an idea have a look at an example.

Chapter 6

Contents

Move Properties	B,KO, MN, W
Setup Properties	AB,AE, AW, PL
Node Annotation Properties	C,DM, GB,GW, HO,N, UC, V
Move Annotation Properties	BM,DO, IT, TE
Markup Properties	AR,CR, DD,LB, LN,MA, SL,SQ, TR
Root Properties	AP,CA, FF,GM, ST, SZ
Game Info Properties	AN,BR, BT, CP,DT, EV, GN,GC, ON, OT,PB, PC, PW,RE, RO, RU,SO, TM, US,WR, WT
Timing Properties	BL,OB, OW, WL
Miscellaneous Properties	FG,PM, VW

Chapter 7

SGF Properties (FF[4])

7.1 Move properties

Property: B
Propvalue: move
Propertytype: move
Function: Execute a black move. This is one of the most used properties in actual collections. As long as the given move is syntactically correct it should be executed. It doesn't matter if the move itself is illegal (e.g. recapturing a ko in a Go game). Have a look at how to execute a Go-move. B and W properties must not be mixed within a node.
Related: W, KO

Property: KO
Propvalue: none
Propertytype: move
Function: Execute a given move (B or W) even it's illegal. This is an optional property, SGF viewers themselves should execute ALL moves. It's purpose is to make it easier for other applications (e.g. computer-players) to deal with illegal moves. A KO property without a black or white move within the same node is illegal.
Related: W, B

Property: MN
Propvalue: number
Propertytype: move
Function: Sets the move number to the given value, i.e. a move specified in this node has exactly this move-number. This can be useful for variations or printing.
Related: B, W, FG, PM

Property: W
Propvalue: move
Propertytype: move
Function: Execute a white move. This is one of the most used properties in actual collections. As long as the given move is syntactically correct it should be executed. It doesn't matter if the move itself is illegal (e.g. recapturing a ko in a Go game). Have a look at how to execute a Go-move. B and W properties must not be mixed within a node.
Related: B, KO

7.2 Setup properties

7.2.1 Restrictions

AB, AW and AE must have unique points, i.e. it is illegal to place different colors on the same point within one node.

AB, AW and AE values which don't change the board, e.g. placing a black stone with AB[] over a black stone that's already there, is bad style. Applications may want to delete these values and issue a warning.

Property: AB
Propvalue: list of stone
Propertytype: setup
Function: Add black stones to the board. This can be used to set up positions or problems. Adding is done by 'overwriting' the given point with black stones. It doesn't matter what was there before. Adding a stone doesn't make any prisoners nor any other captures (e.g. suicide). Thus it's possible to create illegal board positions. Points used in stone type must be unique.
Related: AW, AE, PL

Property: AE
Propvalue: list of point
Propertytype: setup
Function: Clear the given points on the board. This can be used to set up positions or problems. Clearing is done by 'overwriting' the given points, so that they contain no stones. It doesn't matter what was there before. Clearing doesn't count as taking prisoners. Points must be unique.
Related: AB, AW, PL

Property: AW

Propvalue: list of stone
 Propertytype: setup
 Function: Add white stones to the board. This can be used to set up positions or problems. Adding is done by 'overwriting' the given points with white stones. It doesn't matter what was there before. Adding a stone doesn't make any prisoners nor any other captures (e.g. suicide). Thus it's possible to create illegal board positions.
 Points used in stone type must be unique.
 Related: AB, AE, PL

Property: PL
 Propvalue: color
 Propertytype: setup
 Function: PL tells whose turn it is to play. This can be used when setting up positions or problems.
 Related: AE, AB, AW

7.3 Node annotation properties

Property: C
 Propvalue: text
 Propertytype: -
 Function: Provides a comment text for the given node. The purpose of providing both a node name and a comment is to have a short identifier like "doesn't work" or "Dia. 15" that can be displayed directly with the properties of the node, even if the comment is turned off or shown in a separate window.
 See text-type for more info.
 Related: N, ST, V, UC, DM, HO

Property: DM
 Propvalue: double
 Propertytype: -
 Function: The position is even. SGF viewers should display a message. This property may indicate main variations in opening libraries (joseki) too. Thus DM[2] indicates an even result for both players and that this is a main variation of this joseki/opening.
 This property must not be mixed with UC, GB or GW within a node.
 Related: UC, GW, GB

Property: GB
 Propvalue: double
 Propertytype: -

Function: Something good for black. SGF viewers should display a message. The property is not related to any specific place on the board, but marks the whole node instead.

GB must not be mixed with GW, DM or UC within a node.

Related: GW, C, UC, DM

Property: GW

Propvalue: double

Propertytype: -

Function: Something good for white. SGF viewers should display a message. The property is not related to any specific place on the board, but marks the whole node instead.

GW must not be mixed with GB, DM or UC within a node.

Related: GB, C, UC, DM

Property: H0

Propvalue: double

Propertytype: -

Function: Node is a 'hotspot', i.e. something interesting (e.g. node contains a game-deciding move).

SGF viewers should display a message.

The property is not related to any specific place on the board, but marks the whole node instead.

Sophisticated applications could implement the navigation command next/previous hotspot.

Related: GB, GW, C, UC, DM

Property: N

Propvalue: simpletext

Propertytype: -

Function: Provides a name for the node. For more info have a look at the C-property.

Related: C, ST, V

Property: UC

Propvalue: double

Propertytype: -

Function: The position is unclear. SGF viewers should display a message. This property must not be mixed with DM, GB or GW within a node.

Related: DM, GW, GB

Property: V

Propvalue: real

Propertytype: -

Function: Define a value for the node. Positive values are good for black, negative values are good for white.

The interpretation of particular values is game-specific.

In Go, this is the estimated score.
Related: C, N, RE

7.4 Move annotation properties

7.4.1 Restrictions

Move annotation properties without a move (B[] or W[]) within the same node are senseless and therefore illegal. Applications should delete such properties and issue a warning.

BM, TE, DO and IT are mutual exclusive, i.e. they must not be mixed within a single node.

Property: BM
Propvalue: double
Propertytype: move
Function: The played move is bad.
Viewers should display a message.
Related: TE, DO, IT

Property: DO
Propvalue: none
Propertytype: move
Function: The played move is doubtful.
Viewers should display a message.
Related: BM, TE, IT

Property: IT
Propvalue: none
Propertytype: move
Function: The played move is interesting.
Viewers should display a message.
Related: BM, DO, TE

Property: TE
Propvalue: double
Propertytype: move
Function: The played move is a tesuji (good move).
Viewers should display a message.
Related: BM, DO, IT

7.5 Markup properties

7.5.1 Restrictions

CR, MA, SL, SQ and TR points must be unique, i.e. it's illegal to have two or more of these markups on the same point within a node.

Property: AR
 Propvalue: list of composed point ':' point
 Propertytype: -
 Function: Viewers should draw an arrow pointing FROM the first point
 TO the second point.
 It's illegal to specify the same arrow twice,
 e.g. (Go) AR[aa:bb][aa:bb]. Different arrows may have the same
 starting or ending point though.
 It's illegal to specify a one point arrow, e.g. AR[cc:cc]
 as it's impossible to tell into which direction the
 arrow points.
 Related: TR, CR, LB, SL, MA, SQ, LN

Property: CR
 Propvalue: list of point
 Propertytype: -
 Function: Marks the given points with a circle.
 Points must be unique.
 Related: TR, MA, LB, SL, AR, SQ, LN

Property: DD
 Propvalue: elist of point
 Propertytype: inherit
 Function: Dim (grey out) the given points.
 Have a look at the picture to get an idea.
 DD[] clears any setting, i.e. it undims everything.
 Related: VW

Property: LB
 Propvalue: list of composed point ':' simpletext
 Propertytype: -
 Function: Writes the given text on the board. The text should be
 centered around the given point. Note: there's no longer
 a restriction to the length of the text to be displayed.
 Have a look at the FF4 example file on possibilities
 to display long labels (pictures five and six).
 Points must be unique.
 Related: TR, CR, MA, SL, AR, SQ, LN

Property: LN
 Propvalue: list of composed point ':' point
 Propertytype: -
 Function: Applications should draw a simple line from one point
 to the other.
 It's illegal to specify the same line twice,
 e.g. (Go) LN[aa:bb][aa:bb]. Different lines may have the same
 starting or ending point though.
 It's illegal to specify a one point line, e.g. LN[cc:cc].

Related: TR, CR, MA, SL, AR, SQ, LB

Property: MA

Propvalue: list of point

Propertytype: -

Function: Marks the given points with an 'X'.

Points must be unique.

Related: TR, CR, LB, SL, AR, SQ, LN

Property: SL

Propvalue: list of point

Propertytype: -

Function: Selected points. Type of markup unknown

(though SGB inverts the colors of the given points).

Points must be unique.

Related: TR, CR, LB, MA, AR, LN

Property: SQ

Propvalue: list of point

Propertytype: -

Function: Marks the given points with a square.

Points must be unique.

Related: TR, CR, LB, SL, AR, MA, LN

Property: TR

Propvalue: list of point

Propertytype: -

Function: Marks the given points with a triangle.

Points must be unique.

Related: MA, CR, LB, SL, AR, LN

7.6 Root properties

Property: AP

Propvalue: composed simpletext ":" simpletext

Propertytype: root

Function: Provides the name and version number of the application used to create this gametree.

The name should be unique and must not be changed for different versions of the same program.

The version number itself may be of any kind, but the format used must ensure that by using an ordinary string-compare, one is able to tell if the version is lower or higher than another version number.

Here's the list of known applications and their names:

Application	System	Name
[CGoban:1.6.2]	Unix	CGoban
[Hibiscus:2.1]	Windows 95	Hibiscus Go Editor
[IGS:5.0]	Internet	Go Server
[Many Faces of Go:10.0]	Windows 95	The Many Faces of Go
[MGT:?]	DOS/Unix	MGT
[NNGS:?]	Unix	No Name Go Server
[Primiview:3.0]	Amiga OS3.0	Primiview
[SGB:?]	Macintosh	Smart Game Board
[SmartGo:1.0]	Windows	SmartGo

Related: FF, GM, SZ, ST, CA

Property: CA

Propvalue: simpletext

Propertytype: root

Function: Provides the used charset for SimpleText and Text type.

Default value is 'ISO-8859-1' aka 'Latin1'.

Only charset names (or their aliases) as specified in RFC 1345 (or updates thereof) are allowed.

Basically this field uses the same names as MIME messages in their 'charset=' field (in Content-Type).

RFC's can be obtained via FTP from DS.INTERNIC.NET, NIS.NSF.NET, WUARCHIVE.WUSTL.EDU, SRC.DOC.IC.AC.UK or FTP.IMAG.FR.

Related: FF, C, text type

Property: FF

Propvalue: number (range: 1-4)

Propertytype: root

Function: Defines the used file format. For difference between those formats have a look at the history of SGF.

Default value: 1

Applications must be able to deal with different file formats within a collection.

Related: GM, SZ, ST, AP, CA

Property: GM

Propvalue: number (range: 1-16)

Propertytype: root

Function: Defines the type of game, which is stored in the current gametree. The property should help applications to reject games, they cannot handle.

Valid numbers are: Go = 1, Othello = 2, chess = 3,

Gomoku+Renju = 4, Nine Men's Morris = 5, Backgammon = 6,

Chinese chess = 7, Shogi = 8, Lines of Action = 9,

Ataxx = 10, Hex = 11, Jungle = 12, Neutron = 13,

Philosopher's Football = 14, Quadrature = 15, Trax = 16,
Tantrix = 17, Amazons = 18, Octi = 19, Gess = 20,
Twixt = 21, Zertz = 22, Plateau = 23, Yinsh = 24,
Punct = 25, Gobblet = 26, hive = 27, Exxit = 28,
Hnefatale = 29, Kuba = 30, Tripples = 31, Chase = 32,
Tumbling Down = 33, Sahara = 34, Byte = 35, Focus = 36,
Dvonn = 37, Tamsk = 38, Gipf = 39, Kropki = 40.
Default value: 1

Different kind of games may appear within a collection.

Related: FF, SZ, ST, AP, CA

Property: ST

Propvalue: number (range: 0-3)

Propertytype: root

Function: Defines how variations should be shown (this is needed to synchronize the comments with the variations). If ST is omitted viewers should offer the possibility to change the mode online. Basically most programs show variations in two ways: as markup on the board (if the variation contains a move) and/or as a list (in a separate window).

The style number consists two options.

- 1) show variations of successor node (children) (value: 0)
show variations of current node (siblings) (value: 1)
affects markup & list
- 2) do board markup (value: 0)
no (auto-) board markup (value: 2)
affects markup only.

Using no board markup could be used in problem collections or if variations are marked by subsequent properties. Viewers should take care, that the automatic variation board markup DOESN'T overwrite any markup of other properties.

The final number is calculated by adding the values of each option. Example: 3 = no board markup/variations of current node
1 = board markup/variations of current node

Default value: 0

Related: C, FF, GM, SZ, AP, CA

Property: SZ

Propvalue: (number | composed number ':' number)

Propertytype: root

Function: Defines the size of the board. If only a single value is given, the board is a square; with two numbers given, rectangular boards are possible. If a rectangular board is specified, the first number specifies the number of columns, the second provides the number of rows. Square boards must not be defined using the compose type value: e.g. SZ[19:19] is illegal.

The valid range for SZ is any size greater or equal to 1x1.
For Go games the maximum size is limited to 52x52.

Default value: game specific
for Go: 19 (square board)
for Chess: 8 (square board)

Different board sizes may appear within a collection.

See move-/point-type for more info.

Related: FF, GM, ST, AP, CA

7.7 Game info properties

Property: AN
Propvalue: simpletext
Propertytype: game-info
Function: Provides the name of the person, who made the annotations
to the game.
Related: US, SO, CP

Property: BR
Propvalue: simpletext
Propertytype: game-info
Function: Provides the rank of the black player.
For Go (GM[1]) the following format is recommended:
"...k" or "...kyu" for kyu ranks and
"...d" or "...dan" for dan ranks.
Go servers may want to add '?' for an uncertain rating and
'*' for an established rating.

Related: PB, BT, WR

Property: BT
Propvalue: simpletext
Propertytype: game-info
Function: Provides the name of the black team, if game was part of a
team-match (e.g. China-Japan Supermatch).
Related: PB, PW, WT

Property: CP
Propvalue: simpletext
Propertytype: game-info
Function: Any copyright information (e.g. for the annotations) should
be included here.
Related: US, SO, AN

Property: DT
Propvalue: simpletext
Propertytype: game-info

Function: Provides the date when the game was played.
It is MANDATORY to use the ISO-standard format for DT.
Note: ISO format implies usage of the Gregorian calendar.
Syntax:
"YYYY-MM-DD" year (4 digits), month (2 digits), day (2 digits)
Do not use other separators such as "/", " ", ",", or ".".
Partial dates are allowed:
"YYYY" - game was played in YYYY
"YYYY-MM" - game was played in YYYY, month MM
For games that last more than one day: separate other dates
by a comma (no spaces!); following shortcuts may be used:
"MM-DD" - if preceded by YYYY-MM-DD, YYYY-MM, MM-DD, MM or DD
"MM" - if preceded by YYYY-MM or MM
"DD" - if preceded by YYYY-MM-DD, MM-DD or DD
Shortcuts acquire the last preceding YYYY and MM (if
necessary).
Note: interpretation is done from left to right.
Examples:
1996-05,06 = played in May,June 1996
1996-05-06,07,08 = played on 6th,7th,8th May 1996
1996,1997 = played in 1996 and 1997
1996-12-27,28,1997-01-03,04 = played on 27th,28th
of December 1996 and on 3rd,4th January 1997
Note: it's recommended to use shortcuts whenever possible,
e.g. 1997-05-05,06 instead of 1997-05-05,1997-05-06
Related: EV, RO, PC, RU, RE, TM

Property: EV
Propvalue: simpletext
Propertytype: game-info
Function: Provides the name of the event (e.g. tournament).
Additional information (e.g. final, playoff, ..)
shouldn't be included (see RO).
Related: GC, RO, DT, PC, RU, RE, TM

Property: GN
Propvalue: simpletext
Propertytype: game-info
Function: Provides a name for the game. The name is used to
easily find games within a collection.
The name should therefore contain some helpful information
for identifying the game. 'GameName' could also be used
as the file-name, if a collection is split into
single files.
Related: GC, EV, DT, PC, RO

Property: GC
Propvalue: text

Propertytype: game-info
Function: Provides some extra information about the following game.
The intend of GC is to provide some background information
and/or to summarize the game itself.
Related: GN, ON, AN, CP

Property: ON
Propvalue: simpletext
Propertytype: game-info
Function: Provides some information about the opening played
(e.g. san-ren-sei, Chinese fuseki, etc.).
Related: GN, GC

Property: OT
Propvalue: simpletext
Propertytype: game-info
Function: Describes the method used for overtime (byo-yomi).
Examples: "5 mins Japanese style, 1 move / min",
"25 moves / 10 min".
Related: TM, BL, WL, OB, OW

Property: PB
Propvalue: simpletext
Propertytype: game-info
Function: Provides the name of the black player.
Related: PW, BT, WT

Property: PC
Propvalue: simpletext
Propertytype: game-info
Function: Provides the place where the games was played.
Related: EV, DT, RO, RU, RE, TM

Property: PW
Propvalue: simpletext
Propertytype: game-info
Function: Provides the name of the white player.
Related: PB, BT, WT

Property: RE
Propvalue: simpletext
Propertytype: game-info
Function: Provides the result of the game. It is MANDATORY to use the
following format:
"0" (zero) or "Draw" for a draw (jigo),
"B+" ["score"] for a black win and
"W+" ["score"] for a white win
Score is optional (some games don't have a score e.g. chess).

If the score is given it has to be given as a real value,
e.g. "B+0.5", "W+64", "B+12.5"
Use "B+R" or "B+Resign" and "W+R" or "W+Resign" for a win by
resignation. Applications must not write "Black resigns".
Use "B+T" or "B+Time" and "W+T" or "W+Time" for a win on time,
"B+F" or "B+Forfeit" and "W+F" or "W+Forfeit" for a win by
forfeit,
"Void" for no result or suspended play and
"?" for an unknown result.

Related: EV, DT, PC, RO, RU, TM

Property: RO
Propvalue: simpletext
Propertytype: game-info
Function: Provides round-number and type of round. It should be
written in the following way: RO[xx (tt)], where xx is the
number of the round and (tt) the type:
final, playoff, league, ...
Related: EV, DT, PC, RU, RE, TM

Property: RU
Propvalue: simpletext
Propertytype: game-info
Function: Provides the used rules for this game.
Because there are many different rules, SGF requires
mandatory names only for a small set of well known rule sets.
Note: it's beyond the scope of this specification to give an
exact specification of these rule sets.
Mandatory names for Go (GM[1]):
"AGA" (rules of the American Go Association)
"GOE" (the Ing rules of Goe)
"Japanese" (the Nihon-Kiin rule set)
"NZ" (New Zealand rules)

Related: EV, DT, PC, RO, RE, TM

Property: SO
Propvalue: simpletext
Propertytype: game-info
Function: Provides the name of the source (e.g. book, journal, ...).
Related: US, AN, CP

Property: TM
Propvalue: real
Propertytype: game-info
Function: Provides the time limits of the game.
The time limit is given in seconds.

Related: EV, DT, PC, RO, RU, RE

Property: US

Propvalue: simpletext

Propertytype: game-info

Function: Provides the name of the user (or program), who entered the game.

Related: SO, AN, CP

Property: WR

Propvalue: simpletext

Propertytype: game-info

Function: Provides the rank of the white player. For recommended format see BR.

Related: PW, WT, BR

Property: WT

Propvalue: simpletext

Propertytype: game-info

Function: Provide the name of the white team, if game was part of a team-match (e.g. China-Japan Supermatch).

Related: PB, PW, BT

7.8 Timing properties

Property: BL

Propvalue: real

Propertytype: move

Function: Time left for black, after the move was made.
Value is given in seconds.

Related: TM, OT, WL, OB, OW

Property: OB

Propvalue: number

Propertytype: move

Function: Number of black moves left (after the move of this node was played) to play in this byo-yomi period.

Related: TM, OT, BL, WL, OW

Property: OW

Propvalue: number

Propertytype: move

Function: Number of white moves left (after the move of this node was played) to play in this byo-yomi period.

Related: TM, OT, BL, WL, OB

Property: WL
Propvalue: real
Propertytype: move
Function: Time left for white after the move was made.
Value is given in seconds.
Related: TM, OT, BL, OB, OW

7.9 Miscellaneous properties

Property: FG
Propvalue: none | composition of number ":" SimpleText
Propertytype: -
Function: The figure property is used to divide a game into different figures for printing: a new figure starts at the node containing a figure property.
If the value is not empty then

- Simpletext provides a name for the diagram
- Number specifies some flags (for printing).

These flags are:

- coordinates on/off (value: 0/1)
- diagram name on/off (value: 0/2)
- list moves not shown in figure on/off (value: 0/4)
Some moves can't be shown in a diagram (e.g. ko captures in Go) - these moves may be listed as text.
- remove captured stones on/off (value: 0/256)
'remove off' means: keep captured stones in the diagram and don't overwrite stones played earlier - this is the way diagrams are printed in books.
'remove on' means: capture and remove the stones from the display - this is the usual viewer mode.
This flag is specific to Go (GM[1]).
- hoshi dots on/off (value: 0/512)
This flag is specific to Go (GM[1]).
- Ignore flags on/off (value: 32768)
If on, then all other flags should be ignored and the application should use its own defaults.

The final number is calculated by summing up all flag values.
E.g. 515 = coordinates and diagram name off, remove captured stones, list unshown moves, hoshi dots off;
257 = coordinates off, diagram name on, list unshown moves, don't remove captured stones, hoshi dots on.
(this is how diagrams are printed in e.g. Go World)

Note: FG combined with VW, MN and PM are mighty tools to print and compile diagrams.

Related: MN, PM, VW

Property: PM
Propvalue: number
Propertytype: inherit
Function: This property is used for printing.
It specifies how move numbers should be printed.
0 ... don't print move numbers
1 ... print move numbers as they are
2 ... print 'modulo 100' move numbers
This mode is usually used in books or magazines.
Note: Only the first move number is calculated
'modulo 100' and the obtained number is increased
for each move in the diagram.
E.g. A figure containing moves
32-78 is printed as moves 32-78
102-177 is printed as moves 2-77
67-117 is printed as moves 67-117
154-213 is printed as moves 54-113
Default value: 1
Related: MN, FG

Property: VW
Propvalue: elist of point
Propertytype: inherit
Function: View only part of the board. The points listed are
visible, all other points are invisible.
Note: usually the point list is given in compressed
format (see 'point' type)!
Points have to be unique.
Have a look at the picture to get an idea.
VW[] clears any setting, i.e. the whole board is
visible again.
Related: DD, PM, FG

Chapter 8

Variations

SGF's internal structure is a tree of property lists. This allows storing variations of the main line of play. The tree is written in pre-order. Here's a short sketch for an algorithm for writing a tree in pre-order:

Algorithm	Example tree	pre-order
<pre>
WriteTree(Root)
End<b Write(Node)
 for each child of Node
 WriteTree(child)
 end for
end
 SGF
(;FF[4]C[root] (;C[a] ;C[b] (;C[c])
(;C[d] ;C[e]))
(;C[f] (;C[g] ;C[h] ;C[i])
(;C[j])))
 </pre>		root a b c d e f g h i j

Have a look at the EBNF definition again. Check the SGF example file and the User Guide for more details, such as problems with annotations and variations. Related properties: ST, C, N.

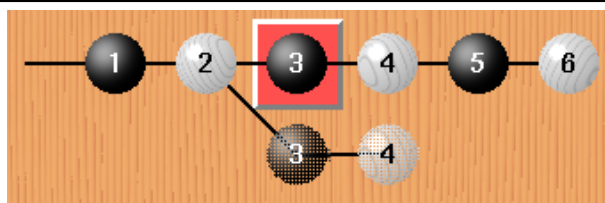
Here are some examples to make it easier to understand the EBNF definition and its application to variations.

Pictures taken from cgoban

No Variation

```
<br>( ;FF[4]GM[1]SZ[19] ;B[aa] ;W[bb] ;B[cc] ;W[dd] ;B[ad] ;W[bd] )<br>
```

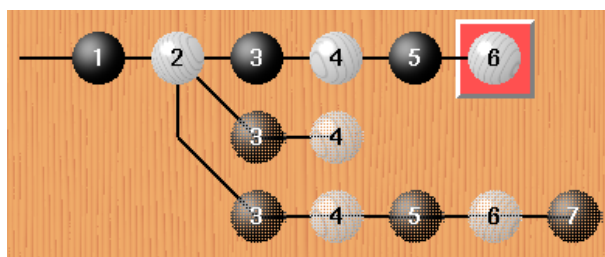




One variation at move 3

```
<br>( ;FF[4]GM[1]SZ[19];B[aa];W[bb]( ;B[cc];W[dd];B[ad];W[bd])<br>( ;B[hh];W[hg]))<br>
```

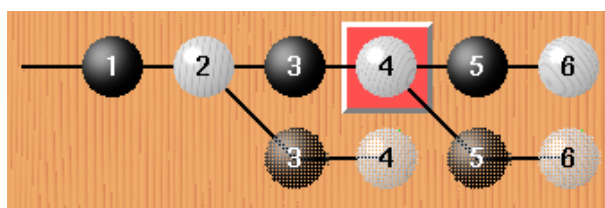
Note the beginning of a new game tree in front of B[cc] and the variation itself on the second line. There are two “)” at the end: one for the variation gametree and one for the end of the main gametreestarting at FF[4].



Two variations at move 3

```
<br>( ;FF[4]GM[1]SZ[19];B[aa];W[bb]( ;B[cc]N[Var
A];W[dd];B[ad];W[bd])<br>( ;B[hh]N[Var
B];W[hg])<br>( ;B[gg]N[Var
C];W[gh];B[hh];W[hg];B[kk]))<br>
```

Usually the main line is labeled variation “A”, the first variation “B”, the second variation “C” and so on. That’s why many annotations refer to the next move as “A”.

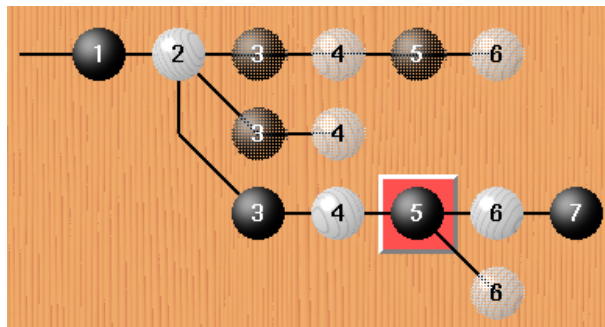


Two variations at different moves

```
<br>( ;FF[4]GM[1]SZ[19];B[aa];W[bb]( ;B[cc];W[dd]( ;B[ad];W[bd])<br>( ;B[ee];W[ff]))<br>( ;B[hh];W[gg])
```

The new gametree starts in front of B[ad].

Note that the second variation (2nd line) is in front of the first variation (3rd line). Have a look at the closing brackets as well.



Variation of a variation

```
<br>( ;FF[4]GM[1]SZ[19];B[aa];W[bb]( ;B[cc]N[Var
A];W[dd];B[ad];W[bd])<br>( ;B[hh]N[Var
B];W[hg])<br>( ;B[gg]N[Var
C];W[gh];B[hh] ( ;W[hg]N[Var A];B[kk])
( ;W[kl]N[Var B]))<br>
```

The new game tree starts in front of W[hg].

Note that there are three “)” at the end: one for the W[kl] variation, one for the B[gg] variation, and one for the main gametree.

8.1 Common pitfalls:

Every variation has at least one node (see EBNF\ definition)! That is, the smallest possible variation looks like (;) - () is an error! Another example: removing all properties from the “variation of a variation” example leads to: (; ; ; (; ; ;) (; ;) (; ; ;) (; ; ;)) and additionally removing unnecessary nodes leads to (; (;) (;) (; (;) (; ;)))

Properties are part of a node, therefore (W[tt]) is an error. Correct is (; W[tt])

No properties outside a gametree! E.g. (;)W[tt] is an error.

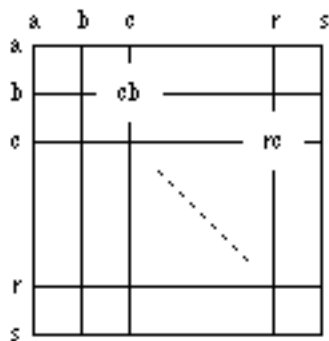
Chapter 9

Go (GM[1])

9.1 Move, Point & Stone type

In Go the Stone becomes Point and

the Move and Point type are the same: two lowercase letters.



Coordinate system for points and moves

The first letter designates the column (left to right), the second the row (top to bottom). The upper left part of the board is used for smaller boards, e.g. letters “a”-“m” for 13*13.

A pass move is shown as ‘[]’ or alternatively as ‘[tt]’ (only for boards $\leq 19 \times 19$), i.e. applications should be able to deal with both representations. ‘[tt]’ is kept for compatibility with FF[3].

Using lowercase letters only the maximum board size is 26x26.

In FF[4] it is possible to specify board sizes upto 52x52. In this case uppercase letters are used to represent points from 27-52, i.e. ‘a’=1 ... ‘z’=26 , ‘A’=27 ... ‘Z’=52

9.1.1 How to execute a move

When a B (resp. W) property is encountered, a stone of that color is placed on the given position (no matter what was there before).

Then the application should check any W (resp. B) groups that are adjacent to the stone just placed. If they have no liberties they should be removed and the prisoner count increased accordingly.

Lastly, the B (resp. W) group that the newest stone belongs to should be checked for liberties, and if it has no liberties, it should be removed (suicide) and the prisoner count increased accordingly.

See also: extensive explanation with examples (should leave no question unanswered; is part of FF5 discussion but valid for FF4 as well).

9.2 Properties

TW and TB points must be unique, i.e. it's illegal to list the same point in TB and TW within the same node.

Property: HA
Propvalue: number
Propertytype: game-info
Function: Defines the number of handicap stones (≥ 2).
If there is a handicap, the position should be set up with AB within the same node.
HA itself doesn't add any stones to the board, nor does it imply any particular way of placing the handicap stones.
Related: KM, RE, RU

Property: KM
Propvalue: real
Propertytype: game-info
Function: Defines the komi.
Related: HA, RE, RU

Property: TB
Propvalue: elist of point
Propertytype: -
Function: Specifies the black territory or area (depends on rule set used).
Points must be unique.
Related: TW

Property: TW
Propvalue: elist of point
Propertytype: -
Function: Specifies the white territory or area (depends on rule set used).
Points must be unique.
Related: TB

Chapter 10

FF[4] property index

This is an alphabetical index to all properties defined in FF[4].

New properties are marked with ‘*’, changed properties are marked with ‘!’.

ID	Description	property type	property value
AB	Add Black	setup	list of stone
AE	Add Empty	setup	list of point
AN	Annotation	game-info	simpletext
*AP	Application	root	composed simpletext ':' simpletext
*AR	Arrow	-	list of composed point ':' point
*AS	Who adds stones - (LOA)	-	simpletext
AW	Add White	setup	list of stone
B	Black	move	move
BL	Black time left	move	real
BM	Bad move	move	double
BR	Black rank	game-info	simpletext
BT	Black team	game-info	simpletext
C	Comment	-	text
*CA	Charset	root	simpletext
CP	Copyright	game-info	simpletext
CR	Circle	-	list of point
*DD	Dim points	- (inherit)	elist of point
DM	Even position	-	double
DO	Doubtful	move	none
!DT	Date	game-info	simpletext
EV	Event	game-info	simpletext
FF	Fileformat	root	number (range: 1-4)
!FG	Figure	-	none composed number ":" simpletext
GB	Good for Black	-	double
GC	Game comment	game-info	text
GM	Game	root	number (range: 1-5,7-16)
GN	Game name	game-info	simpletext
GW	Good for White	-	double

HA	Handicap	game-info (Go)	number
HO	Hotspot	-	double
*IP	Initial pos.	game-info (LOA)	simpletext
IT	Interesting	move	none
*IY	Invert Y-axis	game-info (LOA)	simpletext
KM	Komi	game-info (Go)	real
KO	Ko	move	none
!LB	Label	-	list of composed point ':' simpletext
*LN	Line	-	list of composed point ':' point
MA	Mark	-	list of point
MN	set move number	move	number
N	Nodename	-	simpletext
OB	OtStones Black	move	number
ON	Opening	game-info	simpletext
*OT	Overtime	game-info	simpletext
OW	OtStones White	move	number
PB	Player Black	game-info	simpletext
PC	Place	game-info	simpletext
PL	Player to play	setup	color
*PM	Print move mode	- (inherit)	number
PW	Player White	game-info	simpletext
!RE	Result	game-info	simpletext
RO	Round	game-info	simpletext
!RU	Rules	game-info	simpletext
*SE	Markup	- (LOA)	point
SL	Selected	-	list of point
SO	Source	game-info	simpletext
*SQ	Square	-	list of point
*ST	Style	root	number (range: 0-3)
*SU	Setup type	game-info (LOA)	simpletext
!SZ	Size	root	(number composed number ':' number)
TB	Territory Black	- (Go)	elist of point
TE	Tesuji	move	double
TM	Timelimit	game-info	real
TR	Triangle	-	list of point
TW	Territory White	- (Go)	elist of point
UC	Unclear pos	-	double
US	User	game-info	simpletext
V	Value	-	real
*VW	View	- (inherit)	elist of point
W	White	move	move
WL	White time left	move	real
WR	White rank	game-info	simpletext
WT	White team	game-info	simpletext

Chapter 11

FF[4] property index

This is an alphabetical index to all properties defined in FF[4] sorted by property type.

New properties are marked with ‘*’, changed properties are marked with ‘!’.

11.1 General properties

ID	Description	property type	property value
-----	-----	-----	-----
B	Black	move	move
BL	Black time left	move	real
BM	Bad move	move	double
DO	Doubtful	move	none
IT	Interesting	move	none
KO	Ko	move	none
MN	set MoveNumber	move	number
OB	OtStones Black	move	number
OW	OtStones White	move	number
TE	Tesuji	move	double
W	White	move	move
WL	White time left	move	real
AB	Add Black	setup	list of stone
AE	Add Empty	setup	list of point
AW	Add White	setup	list of stone
PL	Player to play	setup	color
*AR	Arrow	-	list of composed point ':' point
C	Comment	-	text
CR	Circle	-	list of point
*DD	Dim points	- (inherit)	elist of point
DM	Even position	-	double
!FG	Figure	-	none composed number ":" simpletext
GB	Good for Black	-	double

GW	Good for White	-	double
HO	Hotspot	-	double
!LB	Label	-	list of composed point ':' simpletext
*LN	Line	-	list of composed point ':' point
MA	Mark	-	list of point
N	Nodename	-	simpletext
*PM	Print move mode	-(inherit)	number
SL	Selected	-	list of point
*SQ	Square	-	list of point
TR	Triangle	-	list of point
UC	Unclear pos	-	double
V	Value	-	real
*VW	View	-(inherit)	elist of point
*AP	Application	root	composed simpletext ':' number
*CA	Charset	root	simpletext
FF	Fileformat	root	number (range: 1-4)
GM	Game	root	number (range: 1-5,7-17)
*ST	Style	root	number (range: 0-3)
!SZ	Size	root	(number composed number ':' number)
AN	Annotation	game-info	simpletext
BR	Black rank	game-info	simpletext
BT	Black team	game-info	simpletext
CP	Copyright	game-info	simpletext
!DT	Date	game-info	simpletext
EV	Event	game-info	simpletext
GC	Game comment	game-info	text
GN	Game name	game-info	simpletext
ON	Opening	game-info	simpletext
*OT	Overtime	game-info	simpletext
PB	Player Black	game-info	simpletext
PC	Place	game-info	simpletext
PW	Player White	game-info	simpletext
!RE	Result	game-info	simpletext
RO	Round	game-info	simpletext
!RU	Rules	game-info	simpletext
SO	Source	game-info	simpletext
TM	Timelimit	game-info	real
US	User	game-info	simpletext
WR	White rank	game-info	simpletext
WT	White team	game-info	simpletext

11.2 Go (GM[1]) specific properties

ID	Description	property type	property value

TB	Territory Black -		elist of point
TW	Territory White -		elist of point
HA	Handicap	game-info	number
KM	Komi	game-info	real

11.3 Lines of Action (GM[9]) specific properties

ID	Description	property type	property value
*AS	Who adds stones -		simpletext
*IP	Initial pos.	game-info	simpletext
*IY	Invert Y-axis	game-info	simpletext
*SE	Markup	-	point
*SU	Setup type	game-info	simpletext

Chapter 12

Property index of FF[1]-FF[4]

This is an alphabetical index to all properties defined in FF[1] (as in Kierulf's thesis), FF[3] (as on Martin's pages) and FF[4] (as in this spec).

Note: FF[2] was never made public. It's more or less identical to FF[1] - the only changes known (to me) are that the BS/WS values had been redefined.

ID	FF	Description	property type	property value
--	----	-----	-----	-----
AB	1234	Add Black	setup	list of stone
AE	1234	Add Empty	setup	list of point
AN	--34	Annotation	game-info	simpletext
AP	---4	Application	root	composed simpletext : simpletext
AR	---4	Arrow	-	list of composed point : point
AS	---4	Who adds stones - (LOA)	-	simpletext
AW	1234	Add White	setup	list of stone
B	1234	Black	move	move
BL	1234	Black time left	move	real
BM	1234	Bad move	move	double
BR	1234	Black rank	game-info	simpletext
BS	123-	Black species	game-info	number
BT	--34	Black team	game-info	simpletext
C	1234	Comment	-	text
CA	---4	Charset	root	simpletext
CH	123-	Check mark	-	double
CP	--34	Copyright	game-info	simpletext
CR	--34	Circle	-	list of point
DD	---4	Dim points	- (inherit)	elist of point
DM	--34	Even position	-	double
DO	--34	Doubtful	move	none
DT	1234	Date	game-info	simpletext
EL	12--	Eval. comp move	-	number
EV	1234	Event	game-info	simpletext
EX	12--	Expected move	-	move
FF	-234	Fileformat	root	number (range: 1-4)

FG	1234	Figure	-	none composed number : simpletext
GB	1234	Good for Black	-	double
GC	1234	Game comment	game-info	text
GM	1234	Game	root	number (range: 1-5,7-16)
GN	1234	Game name	game-info	simpletext
GW	1234	Good for White	-	double
HA	1234	Handicap	game-info (Go)	number
HO	--34	Hotspot	-	double
ID	--3-	Game identifier	game-info	simpletext
IP	---4	Initial pos.	game-info (LOA)	simpletext
IT	--34	Interesting	move	none
IY	---4	Invert Y-axis	game-info (LOA)	simpletext
KM	1234	Komi	game-info (Go)	real
KO	--34	Ko	move	none
L	12--	Letters	-	list of point
LB	--34	Label	-	list of composed point : simpletext
LN	--34	Line	-	list of composed point : point
LT	--3-	Lose on time	-	none
M	12--	Simple markup	-	list of point
MA	--34	Mark with X	-	list of point
MN	--34	Set move number	move	number
N	1234	Nodename	-	simpletext
OB	--34	OtStones Black	move	number
OM	--3-	Moves/overtime	-	number
ON	--34	Opening	game-info	simpletext
OP	--3-	Overtime length	-	real
OT	---4	Overtime	game-info	simpletext
OV	--3-	Operator overhead	-	real
OW	--34	OtStones White	move	number
PB	1234	Player Black	game-info	simpletext
PC	1234	Place	game-info	simpletext
PL	1234	Player to play	setup	color
PM	---4	Print move mode	- (inherit)	number
PW	1234	Player White	game-info	simpletext
RE	1234	Result	game-info	simpletext
RG	123-	Region	- (Go)	list of point
RO	1234	Round	game-info	simpletext
RU	--34	Rules	game-info	simpletext
SC	123-	Secure stones	-	list of point
SE	--3-	Selftest moves	-	list of point
SE	---4	Markup	- (LOA)	point
SI	--3-	Sigma	-	double
SL	1234	Selected	-	list of point
SO	1234	Source	game-info	simpletext
SQ	---4	Square	-	list of point
ST	---4	Style	root	number (range: 0-3)
SU	---4	Setup type	game-info (LOA)	simpletext
SZ	1234	Size	root	(number composed number : number)

TB	1234	Territory Black - (Go)		elist of point
TC	--3-	Territory count - (Go)		number
TE	1234	Tesuji	move	double
TM	1234	Timelimit	game-info	real
TR	--34	Triangle	-	list of point
TW	1234	Territory White - (Go)		elist of point
UC	--34	Unclear pos	-	double
US	1234	User	game-info	simpletext
V	1234	Value (score)	-	real
VW	1234	View	- (inherit)	elist of point
W	1234	White	move	move
WL	1234	White time left	move	real
WR	1234	White rank	game-info	simpletext
WS	123-	White species	game-info	number
WT	--34	White team	game-info	simpletext

Chapter 13

Dimmed Stones and View

For more info on the properties have a look at:

- inherit property attribute
- DD property
- VW property

Suppose you have the following SGF-file:

```
(;GM[1]SZ[9]FF[4]
  AB[ac][bc][cc][dc][ec][fc][gc][hc][ic]
  AW[ae][be][ce][de][ee][fe][ge][he][ie]
;DD[aa][ab][ac][ad][ae][af][ag][ah][ai][bi][bh][bg][bf]
  [be][bd][bc][bb][ba][ca][cb][cc][cd][ce]
  VW[aa:bi][ca:ee][ge:ie][ga:ia][gc:ic][gb][ib]
)
```

The same file using compressed point lists for every property:

```
(;GM[1]SZ[9]FF[4]
  AB[ac:ic]AW[ae:ie]
;DD[aa:bi][ca:ce]
  VW[aa:bi][ca:ee][ge:ie][ga:ia][gc:ic][gb][ib]
)
```

13.1 Picture of node 0

13.2 Picture of node 1

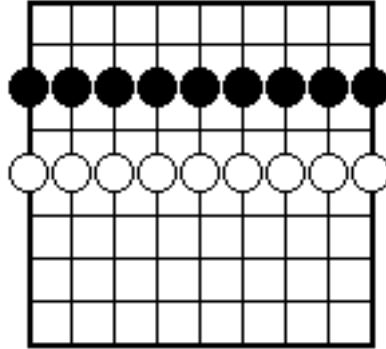


Figure 13.1: [board: node 0 (dim1.gif)]

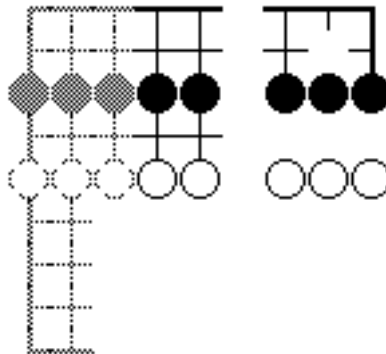


Figure 13.2: [board: node 1, using DD & VW (dim2.gif)]

Chapter 14

SGF FF[4] History

14.0.1 2021-12-01

- Twixt
 - added specification for long moves
 - adapted move syntax a bit
 - other minor corrections
- Hex
 - add *pass* move
 - reduce minimum board size to 1x1; allow arbitrarily large boards
 - define point syntax for more than 26 columns

14.0.2 2007 - 2016

History between 2007-2016 is missing here; changes were mostly related to

- reserving new game numbers,
- new releases of SGFC, and
- adding a draft specification for the Uniform Type Identifier (UTI) on 2013-05-05.

14.0.3 2006-08-06

- Added link to CA[] property in sections about Text/SimpleText property value types.
- New SGFC release: V1.16

14.0.4 2006-06-25

- Added new game number by request of Dave Dyer: hive = 27

14.0.5 2006-05-13

- Added new game number by request of Dave Dyer: Yinsh = 24
- Added new game number by request of Dave Dyer: Punct = 25
- Added new game number by request of Dave Dyer: Gobblet = 26

14.0.6 2002-2005

History between 2002-2005 is missing here; changes were mostly related to reserving new game numbers, and adding specifications for

- Backgammon (GM[6]),
- Lines of Action (GM[9]),
- Hex (GM[11]),
- Amazons (GM[18]),
- Octi (GM[19]),
- Gess (GM[20]),
- Twixt (GM[21]).

14.0.7 2001-08-12

- Some corrections by Anders Kierulf:
- File converting.html: L[] should be converted to uppercase letters instead of lowercase letters.
- File properties.html: Update for SmartGo AP[] entry, changed type of ON[] from Text to SimpleText, corrected typos in “related properties” section for OB[] and VW[].
- File backgammon.html: renamed CP[] property to CO[] (cube owner), added new property DI[] (set dice) - by request of Gary Wong.

14.0.8 2001-01-21

- Added two new games: Octi (GM[19]) and Gess (Gm[20]) by request from Tim Prime.
- New file: gess.html
- New file: octi.html
- File: properties.html: added new game numbers 19, 20 to GM property
- File: sgf4.html: added references to both games

14.0.9 2000-10-18

- Added new game: Backgammon (GM[6]) by request from Gary Wong
- New file: backgammon.html
- File: properties.html: added new game number 6 to GM property
- File: sgf4.html: added references to Backgammon

14.0.10 2000-01-15

- Added “Defaults” and “Properties” sections to Amazons.

14.0.11 2000-01-04

- Added web interface to SGFC
- Made web pages HTML 4.0 compliant

14.0.12 1999-12-17

- Added new game: Amazons (GM[18]) by request from Martin Müller.
- New file: amazons.html
- File: index.html: added references to Amazons
- File: properties.html: added new game number i18 to GM property
- File: sgf4.html: added references to Amazons

14.0.13 1999-01-07

- Created version history file :o)
 - New file var.html: examples for variations
 - File sgf4.html (SGF Basics)
 - Added “Contents” and modified layout
 - Updated link for EBNF dictionary entry
 - Added ... *applications should **not** rely on the order of property values.*
 - Added section “Private Properties”
 - Updated formatting of SimpleText: ... *linebreaks preceded by a "\ " are converted to " ", i.e. they are removed (same as Text type).*
 - Added information for tree structure of SGF file
 - Corrected some syntax errors
 - File properties.html
 - Added “Contents” and modified layout
 - Corrected CS links to “CA”
 - Added new game number by request of Dave Dyer: Tantrix = 17
 - Added flag for FG: *Ignore flags on/off (value: 32768) If on, then all other flags should be ignored and the application should use its own defaults.*
 - Moved ” *TW and TB points must be unique, i.e. it’s illegal to list the same point in TB and TW within the same node.*” to go.html, as TB and TW are described there.
-

Chapter 15

From FF[3] to FF[4]

15.1 General

- Specification is more complete now
- Property identifiers consist only of uppercase (no lowercase in between allowed)
- Go-move/position defined for board sizes upto 52x52
- Rectangular boards possible.
- Compressed point lists for “list of point” PropValues
- Pass move may be specified as ‘[]’
- Text has been divided into two types: simpletext (basically a string) and text, which can be formatted (hard & soft linebreaks).
- Types for properties defined (move, setup, root, gameinfo, none). Therefore nodes can be divided into two types too: move / setup.
- Defined used charset (us-ascii) for property-identifiers and values. For SimpleText and Text charset is specified using CA property (default: ISO-latin-1)
- Property attribute ‘inherit’ defined.
- Time values are given in seconds
- Handling of unknown and illegal properties defined.

15.2 Properties no longer part of FF[4]

- CH (checkmark), SI (sigma), SE (moves in selftest), LT (lose on time), BS, WS, TC: are considered private to SGB
- ID (game-id) : serves no purpose (see specification of EV, RO instead)
- OM (#moves per overtime), OP (length of overtime)
- CI (chinese handicap) : not needed (HA doesn’t imply fixed placement of handicap stones)

- OV (computer type property)
- RG, SC (obsolete markup)

15.3 New properties

- AP (application) : defines writer application
- ST (style) : for handling of comments and variations
- AR (arrow), LN (line), SQ (square) : new board markup
- OT (overtime) : gameinfo to describe byo-yomi rules
- CA (charset) : defines used charset for Text & SimpleText types
- DD (dimmed positons) : grey out the listed positions
- VW (view) : restrict view
- PM (print move) : defines how move numbers should be printed

15.4 Changed properties

- LB (label) may have more than 4 chars now (actually unlimited length)
 - SZ (size) changed (rectangular boards possible now)
 - Format of DT and RE is mandatory now (no longer just recommended)
 - Some rule names are mandatory now.
 - FG may contain some flags and a diagram name
-

Chapter 16

Compatibility of FF[4]

The following list summarizes the compatibility problems of FF[4]. This is done both ways: problems for old (FF[3]) applications when reading a FF[4] file (3 rd 4) and problems of converting FF[3] to FF[4] (3->4). ‘*np*’ is used as abbreviation of ‘*no problem*’ and ‘*nrp*’ is used as abbreviation of ‘*not really a problem*’.

If you find some compatibility issues I’ve missed, please let me know.

16.1 General & Types

- **Verbose property names no longer exist.**
 - 3->4: np (delete all lowercase letters from IDs)
 - 3 rd 4: np
- **Five property types are introduced (and thus 2 node types)**
 - 3->4: FF[3] files may have mixed setup and move-properties in one node. In that case an application should split that node into two nodes: all setup, root and game- info properties and the node name (‘N[]’) into the first node, all other properties into the second.
 - 3 rd 4: np
- **FF[3] text type is split into two types: SimpleText and Text.**
 - 3->4: np
 - 3 rd 4: Soft linebreaks may be treated as hard linebreaks
- **Within the text types anything after a ‘\’ is inserted verbatim.**
 - 3->4: np
 - 3 rd 4: FF[3] only knows some ‘escaped’ chars. It’s uncertain what old applications will do (most likely their routines are already coded in the newly specified way).
- **Rectangular boards and boards >19x19 possible**
 - 3->4: np
 - 3 rd 4: totally incompatible
- **Compressed point lists**
 - 3->4: np
 - 3 rd 4: totally incompatible
- **Time values are in seconds**
 - 3->4: concerning server-games: np, otherwise wrong time is displayed

- 3 rd 4: np
- **Used (default) charset defined**
 - 3->4: some files may contain a charset different from the default. Text will then be unreadable (unless the application allows to switch to another charset manually).
 - 3 rd 4: nrp (as default charset is the one used in FF[3])
- **Pass move may be ‘[]’ now (alternativ to ‘[tt]’)**
 - 3->4: np
 - 3 rd 4: ‘[]’ pass may cause problems (at the least the move will be ignored)

16.2 Properties

- **Obsolete properties**
(CH, SI, SE, LT, ID, OM, OP, CI, OV, BS, WS, SC, RG)
 - 3->4: nrp (properties are treated as unkown properties and are preserved. Information is no longer displayed, though.)
 - 3 rd 4: np
 - **New properties**
(AP, ST, AR, LN, SQ, OT, BO, WO, DD, VW, PM)
 - 3->4: np
 - 3 rd 4: nrp (properties are treated as unkown properties. Information is not displayed though. Some applications may even strip these properties without displaying a warning)
 - **LB may hold more than 4 chars now**
 - 3->4: np
 - 3 rd 4: only 4 chars are displayed/saved (labels get cut)
 - **SZ may hold rectangular boards now**
 - 3->4: np
 - 3 rd 4: incompatible
 - **Format of DT,RE and RU is specfied explicitly**
 - 3->4: unknown formats may appear - should be corrected if possible
 - 3 rd 4: np
 - **FG may contain flags and diagram name**
 - 3->4: np
 - 3 rd 4: nrp (information may get lost though)
-

Chapter 17

Converting old SGF files to FF[4]

17.1 Converting properties

This is a short list of properties that frequently occur in old SGF files and how they should be converted to FF[4].

Property: M
Propvalue: list of point
Function: Simple board markup
Conversion: M should be converted to MA if used on empty board points
and to TR if used on stones.

Property: L
Propvalue: list of point
Function: Label the given points with uppercase letters.
Conversion: L should be converted to LB.
Example: L[fg][es][jk] -> LB[fg:A][es:B][jk:C]

Property: TE then BM within a node
Propvalue: double
Function: Move annotation
Conversion: Should be converted to IT.

Property: BM then TE within a node
Propvalue: double
Function: Move annotation
Conversion: Should be converted to D0.

Property: VW
Propvalue: point, point
Function: Restrict view (just like in FF[4]).
VW wasn't an official property in FF[3] but was used by two
applications (SGB & xgoban)
Conversion: In FF[3] the two given points specified the upper

left and the lower right corner. To convert it to FF[4] either make a compose type value out of it or write the uncompressed point list.

Example: `VW[ba][db] -> VW[ba:db]`
or `-> VW[ba][ca][da][bb][cb][db]`

17.2 Other conversion issues

17.2.1 Property identifiers

In FF[1]-FF[3] lowercase letters were allowed in property identifiers, but in FF[4] only uppercase letters are allowed. I.e. the application **has to** remove all lowercase letters from the file.

Example: *Black[qd];thisisaWhitemove[kk]bbrWyryrerwLerreoi[10.3]* -> *B[qd];W[kk]WL[10.3]*

17.2.2 Property types move & setup

The old fileformats didn't make this distinction, i.e. these files may have mixed setup and move-properties in one node.

In that case an application should split that node into two nodes: all setup, root and game-info properties and the node name ('N[]') into the first node, all other properties into the second. The node name should be in the first node because the node to be split might be the start of a variation.

17.2.3 Illegal & errornous files

There are a lot of corrupt files out there. If you want to write a robust application you should be able to deal with the following cases:

- Line breaks & whitespaces within moves or values: just remove all whitespaces in property values except in text values.
- Empty values e.g. 'DT[]N[]': remove those properties
- Properties without values e.g. 'LB B[aa]': remove the property identifiers
- Properties with too many values e.g. 'B[aa][bb]': this one is critical because it might be caused by a missing '[' e.g. 'B[aa LB[aa][bb]'
- Empty variations e.g. '()': such a construct is illegal because a variation has to have at least one node - remove '()'
- Files not starting with ';' e.g. the ';' is missing - this one is tricky if you want to skip junk in front of the SGF file.
- Faulty game-info values (e.g. RE, DT): the application should correct those values if possible, otherwise leave them as they are.

These are just some frequent errors that occur.

Have a look at the [SGF Syntax Checker & Converter](#) for more sophisticated errors and how to handle them.

Chapter 18

SGF User Guide

Version: 1.2

Author: Arno Hollosi
<ahollosi@xmp.net>Feedback
welcome!Published under the OpenContent
License.

Still unanswered questions?Go to the
Discussion Forum

Basics

1. What is SGF?
2. General Concepts
3. ‘Make a Move’ vs. ‘Place a Stone’
4. Style

More details

5. Variations
6. Board markup
7. Comments and annotations
8. Game Information

Troubleshooting

9. will follow someday
-

Chapter 19

1. What is SGF?

SGF is short for **Smart Game Format**.

SGF is a file format used to store game records of two player board games. It is a text-only tree based format, i.e. it doesn't contain binary data and thus can easily be emailed or posted to newsgroups. Tree based means, that starting from a root one can follow a main path or switch to variations (or variations of variations).

SGF provides many features such as board markup, comments, game information, setup positions etc. In order to create a good SGF file one should have some knowledge of the internal structure of SGF.

19.1 Versions

SGF was invented by Anders Kierulf in 1987 and became more and more popular. Since then SGF has undergone 2 major revisions.

- **FF[1]** is the original specification by Anders Kierulf. This specification is the core of all later versions. Some applications still use this dated version of SGF - e.g. MGT (MS-DOS version) which had become quite popular before Windows became en vogue.
- **FF[3]** was written by Martin Müller in 1993 and was a first step towards a clean specification of the SGF standard. By then SGF was an accepted standard for Go games on the Internet. FF[3] defined a lot of new properties, e.g. many game information properties and some markup properties.
- **FF[4]** was written by Arno Hollosi in 1997 with help from many SGF applications programmers. FF[4] carried on the spirit of FF[3] and provides a clean, unambiguous definition of SGF. New features such as arrows, lines, boards of any size and rectangular boards have been introduced. However as this standard is very young, it isn't widely adopted. This will (hopefully) change in the future.

19.2 What does an SGF file look like?

Here's a short example:

```
(;FF[4]GM[1]SZ[19]AP[SGFC:1.13b]
```

```
PB[troy]BR[12k*]
```

```
PW[john]WR[11k*]
```

```
KM[0.5]RE[W+12.5]
```

```
DT[1998-06-15]
```

```
TM[600]
```

```
;B[pd];W[dp];B[pq];W[dd];B[qk];W[jd];B[fq];W[dj];B[jp];W[jj]
```

```
;B[cn]LB[dn:A][po:B]C[dada: other ideas are 'A' (d6) or 'B' (q5)]
```

```
;W[eo](;B[dl]C[dada: hm - looks troublesome.\
```

```
Usually B plays the 3,3 invasion - see variation];W[qo];B[qp]
```

```
...
```

```
;W[sr];B[sk];W[sg];B[pa];W[gc];B[pi];W[ph];B[de];W[ed];B[kn]
```

```
;W[dh];B[eh];W[se];B[sd];W[af];B[ie];W[id];B[hf];W[hd];B[if]
```

```
;W[fp];B[gq];W[qj];B[sj];W[rh];B[sn];W[so];B[sm];W[ep];B[mn])
```

```
...
```

```
(;W[dq]N[wrong direction];B[qo];W[qp]))
```

19.3 SGF applications

SGF applications are available for all platforms. Most of them are freeware, some are shareware. Have a look at the Go FTP Archive.

As no program is perfect and SGF is evolving it's strongly recommended to update one's favorite SGF application at least once a year. Right now many people still use applications that are over five years old which causes a lot of trouble.

Update your application on a regular basis!

back to top

Chapter 20

2. General Concepts

SGF consists of nodes which are structured in a tree, i.e. a node has exactly one predecessor called parent, but may have one *or more* successors called children. Thus SGF can store game records (a list of moves) and variations of the actual line of play.

A node is the smallest unit visible to the user, i.e. the user steps through the tree node-wise (forward [down the tree], backward [up the tree], etc.).

A node consists of properties. These properties contain a certain kind of information, e.g. the *B* property describes a black move made, the *C* property contains a comment text (don't worry: you don't have to remember the property names :-).

For example: if you step forward and see a new move on the board and a comment in the comment window plus some markup on the board then all this information is represented by **different** properties which are parts of the **same** node.

Thus editing is done in two levels: adding/deleting nodes and adding or deleting properties.

To make it clear: a move is part of a node and not the node part of the move. A move is represented by *one* property but a node may contain *more than one* property.

back to top

Chapter 21

3. ‘Make a Move’ vs. ‘Place a Stone’

SGF provides two ways to add new stones to the board:

- make a move
- place a stone

Making a move is like making a move in a real game, i.e. you can only make moves on empty intersections, you can only make one move per turn (here: per node) and you may take some prisoners by making a move. In most applications the current move is highlighted.

Placing a stone on the board is like setting up a position, e.g. handicap stones, setting up a problem or analysis of positions (“this would work if the position over there would look like this...”). Thus one can place *more than one* stone, stones of different colors, remove stones, replace stones with that of the opposite color all in *one* node.

But: there are no prisoners made as these are not regular moves!

21.1 Restrictions

It is good style (and is required since FF[4]) to distinguish between a move and the position arrived at by this move.

Therefore it’s **illegal** to mix setup properties and move properties within the same node.

Move properties are properties such as a black or white move, annotations on a move (bad move, interesting move, etc.) or how much time a player had left after the current move was made.

Setup properties are properties used to set up or describe a position such as place black/white stones on the board or who’s turn it is to play.

A detailed list of setup and move properties is available.

Unfortunately many applications allow mixing setup and move properties, so it’s up to the user to create a good SGF file.

back to top

Chapter 22

4. Style

- The **first** branch (variation ‘A’ or ‘1’) is the **main** branch.

Variation ‘A’ should always follow the real game. Consider yourself viewing a game where you have to press ‘b’ then ‘c’ and then ‘b’ again just to follow the real game - disturbing.

- Don’t imitate sibling-style variations. Use a sibling-style application instead. Have a look at definition and reason.
- Omit extra pass plays and empty nodes at the end of the game.

The last node of the game should contain the last move on the board. Do not put game information such as ‘Black wins and connects Ko’ into the comment field, rather add another move to the game which connects the Ko.

- Never substitute moves with placing stones.

Never substitute a regular move with placing (setting up) a stone. Some applications rely on regular moves for sophisticated functions. Furthermore applications that show the position of the next move can’t display the position of the setup stone. Have a look at the difference between a real move and setting up a stone.

- Length of labels

Since FF[4] it’s possible to use labels of any size. But some (old) applications have problems displaying labels longer than 2 chars. Use long labels with care!

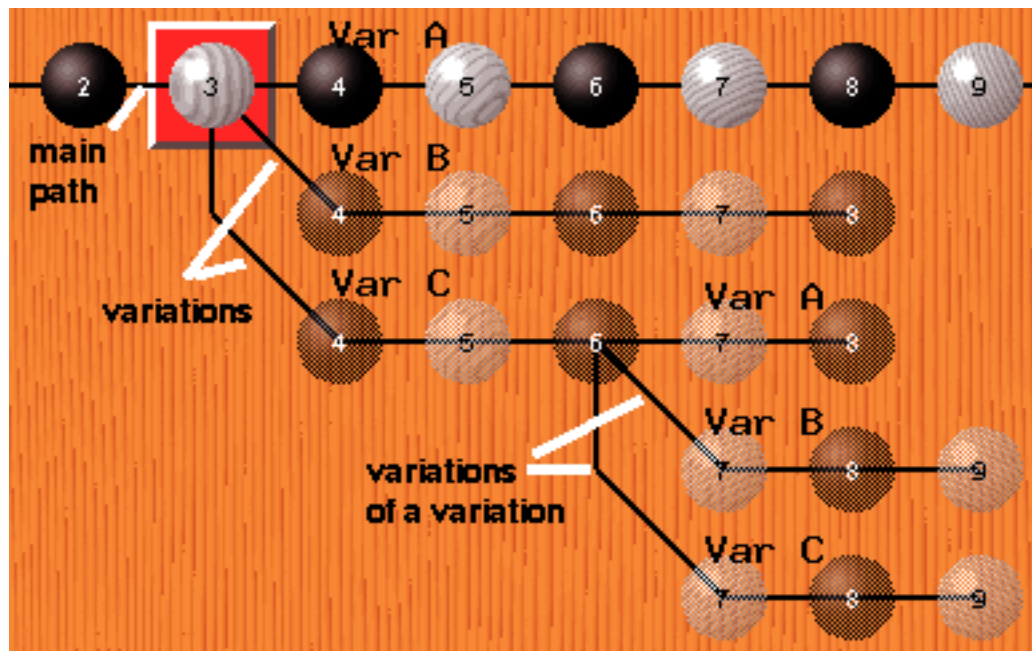
- Don’t store game information such as who’s black/white etc. into the first comment - use game information properties instead. Here’s the reason why.
- Use annotations for standard situations (e.g bad move).

Annotations are represented by extra SGF-properties and thus can be treated in a special way.

[back to top](#)

Chapter 23

5. Variations



SGF allows you to store variations of the main path of play, which is useful for analyzing different lines of play. The picture to the right illustrates this concept. Variations are usually assigned letters starting with 'A' where 'A' is the continuation of the current line of play. That's why some applications refer to the next move as 'A'.

Applications show variations as either siblings or children. **Showing variations as children** means, that if the application is currently at move #3 (like in the picture) it provides you the choice of 'A' through 'C' which are all move #4. That is, by selecting a variation, you step forward in the tree.

Showing variations as siblings means, that the application provides the variation choice at move #4. In this case, selecting a variation selects between moves at the same tree level (here: #4 moves). This method is perceived as alternatives to the current move and by e.g. selecting variation 'B' move #4 'A' is removed from the board and move #4 'B' is shown.

The differences between this two styles may cause confusion when the variations are accompanied by

comments. For example, imagine the comment saying something like: “This is bad. See variation ‘B’ instead.” If the author used a children-style application this comment is stored together with move #3. If readers use a sibling-style application, they see the comment on move #3, but no variations: they appear with the next move. The other case (author: sibling style, reader: children style) has similar implications. Users who read the comment at move #4 have to go back one move and select the variation there.

Most people prefer the sibling style as it seems more natural. They even imitate the sibling style in children-style applications. This is done by removing the previous move and making a new move in the same (first) node of the variation. This is **bad style**. Since FF[4] it’s illegal syntax too. If you have a look at the picture you see that all alternative moves are at the same level (that is, all #4’s are in one column, all #7’s are in one column). By imitating the sibling-style in children-style applications this is no longer the case, as the #4 moves of variation ‘B’ and ‘C’ would appear under move #5 of variation ‘A’. Furthermore those files cannot be converted to other fileformats easily. If you like sibling-style variations then use a sibling-style application!

back to top

Chapter 24

6. Board markup

SGF provides a wide selection of board markup. Almost every markup you see in magazines or books is available in SGF too. However some applications are not able to handle certain types of markup. Here's a short list of markup types available:

Snapshots taken from **cgoban** by W.M. Shubert

Markup	Property	Notes
simple markup	M[]	old (FF[1]), very commonThis markup has been superceeded by MA[] and TR[], however very old applications still use M[] and don't understand MA[] and TR[].
letters	L[]	old (FF[1]), very commonThis markup has beensuperceeded by LB[], however very old applications (e.g. MS-DOS MGT)still use L[] and don't understand LB[]

[back to top](#)

Chapter 25

7. Comments and annotations

In order to comment on a move or position SGF allows to store text in each node, which usually gets displayed in the comment window of your SGF application. Text is easy to edit, but has some disadvantages too:

- language - if you don't understand the language you can't understand the comment
- applications can't use the text to provide more sophisticated functions (such as searching for bad moves or tesujis)
- computer players can't utilize the text information

Therefore SGF provides a set of so-called annotation properties. These properties are encoded differently in the file. They are not stored as readable text, but as tokens, which have a special meaning. Thus the SGF application reading the file knows their meaning and can provide the following:

- multi-lingual support - the application displays the message in the selected language.
- search or other database functions
- computer players can use annotations in their fuseki- or joseki-library to determine the best move or good alternatives.

There are three types of annotation properties: *general annotations*, *move annotations* and *annotations on positions*. Have a look at the following table:

Annotation	Property	Type	may be emphasized?	Meaning
Good for Black	GB	General	yes	Something good for black
Good for White	GW	General	yes	Something good for white
Even position	DM	General	yes	The position is even
Unclear position	UC	General	yes	The position is unclear

Annotation	Property	Type	may be emphasized?	Meaning
Hot spot	HO	General	yes	An important node (e.g. gamedeciding move)
Tesuji	TE	Move	yes	The move played is (locally) a good move
Bad move	BM	Move	yes	The move played is bad
Doubtful move	DO	Move	no	The move played is doubtful
Interesting move	IT	Move	no	The move played is interesting
Black to play	PL	Position	no	It's black turn to play
White to play	PL	Position	no	It's white turn to play

Unfortunately not many applications support these properties yet. This will hopefully change in the future. Consider using annotation properties whenever possible. They have many advantages despite their simplicity.

[back to top](#)

Chapter 26

8. Game Information

SGF provides a wide range of properties to store so called game information. Usually applications provide a dialog or extra window to type in game information.

Note that some entries have a **mandatory format**. Why?

Because standard compliant entries can easily be parsed by applications and therefore can be searched in game collections or displayed in your favorite (customized) scheme. Unfortunately many applications allow the user to enter illegal information - so it's up to you to create correct entries - please take care!

E.g. consider that you've got a game collection of about 5000 pro games and want to look for games played by Cho Chikun in March 1996. Now if dates are stored as e.g. DT[5th March 1996], DT[11/3/96], DT[1996/3/7], DT[1996 6 8] - do you really know which of these games were played in March?

26.1 Why should you use game information?

Sometimes you see a SGF file which has all the information stored into the first comment.

This is bad style!

If the information is stored into a comment it's almost impossible for a computer program to find the relevant data - searching becomes impossible. Therefore store the game information at the proper place - it'll save you much time in the future and makes your database more usable. It makes converting or interchanging of games much more easier too.

26.2 List of game information properties

Here's a complete list with a short description of each item. If this list doesn't answer all your questions have a look at the official, detailed specification.

Note: recommended is not mandatory! But you should use the recommended format whenever possible.

Name	Property	Notes
Black/white name	PB[]/PW[]	Name of the player who played black/white Try to be consistent in using names - for professional players it's suggested to use the same names as Jan van der Steen in his database.
Black/white rank	BR[]/WR[]	Strength of the player who played black/white Recommended format:
"10k" or "10 kyu" for a kyu player
"3d" or "3 dan" for a dan player
 Go servers usually add a `*` (certain rating)
or `?' (uncertain rating) e.g. "10k*"
Black/white team	BT[]/WT[]	Name of the team (for games played in team events)
Result	RE[]	Final result of the game Mandatory format:
"0" (zero) for a draw (jigo)
"B+score" for a black win and
"W+score" for a white win, e.g. "B+2.5", "W+64" or "B+0.5"
"B+R"/"B+Resign" and "W+R"/"W+Resign" for a win by resignation.
 You MUST NOT write "Black resigns"
 A more detailed description is available.
Komi	KM[]	Score adjustment (points added to White's score) Mandatory format:
 Use real values, e.g. "5.5", "0", "0.5" or "-10," etc.
 Don't use: "5 points", "half a point", "5 1/2", etc.

Name	Property	Notes
Handicap	HA[]	Number of handicap stones Mandatory format: Use integer values greater zero, e.g. "1", "5" or "9" Don't use: "2 stones", "three"
Time	TM[]	Regular playing time for each side Mandatory format: Time is given in seconds as a real value, e.g. "4600", "300" Don't use: "1 hour" It's a little bit awkward if your application doesn't transform the real value into a somewhat more human-readable form. But please use real values!
Date	DT[]	Date when game was played Mandatory format: Use the ISO-standard format "YYYY-MM-DD" Do not use other separators such as "/" or " " or ".". Example: a game played on the 5th March 1997 would be encoded as: 1997-03-05 A more detailed description is available.
Event	EV[]	Name of event (e.g. tournament name)
Round	RO[]	Number of tournament round
Place	PC[]	Name of place (e.g. city, country) where game took place
Rules	RU[]	Name of rule set used (e.g. Japanese, Chinese, AGA, GOE, etc.)
Game name	GN[]	Name of the game
Opening	ON[]	Describes the opening played (e.g. san-ren-sei)
Game comment	GC[]	General comment about the game

Name	Property	Notes
Source	SO[]	Name of the source (e.g. book, journal, etc.)
User	US[]	Name of user (or program) who entered the game record
Annotation	AN[]	Name of the person who made the annotations
Copyright	CP[]	Any copyright information

back to top

Chapter 27

1. General SGF FF[4] Example File

27.1 Purpose

The example SGF file and the GIF pictures should illustrate some of the features of SGF FF[4] and give application coders an impression how to draw or represent various properties.

Note: The colors chosen, the screen structure (seperate window for variations & other information, seperate comment window) etc. are specific to Primiview, the application used to create these pictures.

Your application doesn't have to look the same.

You are free to choose whatever colors, structures etc. you want. Same goes for representation of advanced markup such as DD (dimmed points).

27.2 General comments

Primiview's node numbering starts with zero (0) (just like MGT). Node numbering is done in the order the nodes appear in the SGF file.

If your application can't handle SGF collections then you have to extract the second part using a split utility or a text editor.

27.3 Download example SGF file

27.4 Gametree 1: properties

27.4.1 Variation A: (Moves, comments, annotations)

- **Nodes:** 1-13
- **Pictures:**

- ex01.gif: B, W (node 11)

There's nothing special about these nodes.

Note the new (FF[4]) way to write pass moves: “B[]”, “W[]” (nodes 12, 13)

27.4.2 Variation B: (Setup)

- **Nodes:** 14-17
- **Pictures:**
 - ex02.gif: AB,AW (node 14)
 - ex03.gif: AE (node 15)

This variation illustrates how to use AB, AW, AE & PL.

Here compressed point lists are used for the first time. Have a look at the SGF specification and the two pictures to get the idea.

27.4.3 Variation C: (Markup)

- **Nodes:** 18-21
- **Pictures:**
 - ex04.gif: MA, TR, CR, SQ, SL, TB, TW (node 19)
 - ex05.gif: LB (node 20)
 - ex06.gif: LB, different ways of handling long labels (node 20)
 - ex07.gif: AR, LN, DD (node 21)
 - ex08.gif: VW (node 19)

Node 19 shows various kind of board markup.

Node 20 shows labels. Label length are from 1-8 chars. There are two pictures: the second one shows different ways of handling long labels.

Node 21 shows three new FF[4] properties: DD (dimmed points), AR (arrows) and LN (lines)

The picture ex08.gif illustartes how VW (view) works.

The VW property isn't directly encoded into the SGF file. The picture shows node 19 and gives two examples of a restricted view.

27.4.4 Variation D: (Style & text type)

- **Nodes:** 22-32
- **Pictures:**
 - ex09.gif: ST[2] (children / no board markup) (node 23)
 - ex10.gif: ST[1] (siblings / auto-board markup) (node 23)

Node 22 contains a comment text which serves as example for hard and soft linebreaks and for different encodings of linebreaks.

Note that ex09.gif & ex10.gif are both taken from the same node (#23). They only differ in the ST (style) used. The example itself doesn't contain a ST property. The picture just illustrates the use of ST.

27.4.5 Variation E: (Time limits, captures & move numbers)

- **Nodes:** 33-53
- **Pictures:** -

This variation shows the use of BL, WL, OB, OW, MN.

It also contains a suicidal move (and capture).

27.5 Gametree 2: Game-info

This game-tree shows how game-info properties may be stored, if there are several games merged into a single gametree. See the comment texts for what the game-info should look like.

Chapter 28

2. Print Example Files

SGF provides some properties for storing print and layout information. These properties are: FG, VW, PM and MN.

The following files contain two games from GoWorld 78 (of course, the commentary isn't included, but all the board-markup is). You should have this magazine to verify the layout with the example files.

- print1.sgf - 21st Mejin, game 2
 - print2.sgf - 51st Honinbo, game 5
-

Chapter 29

SGF Example Files

The following are raw SGF game records demonstrating FF[4] features.

```
sgf (;FF\[4]AP\[Primiview:3.1]GM\[1]SZ\[19]GN\[Gametree 1: properties]US\[Arno
Hollosi] (;B\[pd]N\[Moves, comments, annotations] C\[Nodename set to: "Moves,
comments, annotations"] ;W\[dp]GW\[1] C\[Marked as "Good for White"] ;B\[pp]GB\[2]
C\[Marked as "Very good for Black"] ;W\[dc]GW\[2] C\[Marked as "Very good for
White"] ;B\[pj]DM\[1] C\[Marked as "Even position"] ;W\[ci]UC\[1] C\[Marked
as "Unclear position"] ;B\[jd]TE\[1] C\[Marked as "Tesuji" or "Good move"] ;W\[jp]BM\[2]
C\[Marked as "Very bad move"] ;B\[gd]DO\[ ] C\[Marked as "Doubtful move"] ;W\[de]IT\[ ]
C\[Marked as "Interesting move"] ;B\[jj] ; C\[White "Pass" move]W\[ ] ; C\[Black
"Pass" move]B\[tt]) (;AB\[dd]\[de]\[df]\[dg]\[do:gq] AW\[jd]\[je]\[jf]\[jg]\[kn:lq]
N\[Setup]C\[Black & white stones at the top are added as single stones.\ \
Black & white stones at the bottom are added using compressed point lists.\
;AE\[ep]\[fp]\[kn]\[lo]\[lq]\[pn:pq] C\[AddEmpty\ \ Black stones & stones
of left white group are erased in FF\[3\\\] way.\ \ White stones at bottom right
were erased using compressed point list.\ ;AB\[pd]AW\[pp]PL\[B]C\[Added two
stones.\ \ Node marked with "Black to play".\ ;PL\[W] C\[Node marked with "White
to play"] ) (;AB\[dd]\[de]\[df]\[dg]\[dh]\[di]\[dj]\[nj]\[ni]\[nh]\[nf]\[ne]\[nd]\
\[gq]\[fq]\[eq]\[dr]\[ds]\[dq]\[dp]\[cp]\[bp]\[ap]\[iq]\[ir]\[is]\[bo]\[bn]\[an]
AW\[pd]\[pe]\[pf]\[pg]\[ph]\[pi]\[pj]\[fd]\[fe]\[ff]\[fh]\[fi]\[fj]\[kh]\[ki]\[ ]
\[oq]\[op]\[pp]\[qp]\[rp]\[sp]\[ro]\[rn]\[sn]\[nq]\[mq]\[lq]\[kq]\[kr]\[ks]\[fs]
\[er]N\[Markup]C\[Position set up without compressed point lists.\ ;TR\[dd]\[de]\[df]\[e]
MA\[dh]\[di]\[dj]\[ej]\[ei]\[eh]\[fh:fj] CR\[nd]\[ne]\[nf]\[od]\[oe]\[of]\[pd:pf]
SQ\[nh]\[ni]\[nj]\[oh]\[oi]\[oj]\[ph:pj] SL\[ih]\[ii]\[ij]\[jj]\[ji]\[jh]\[kh:kj]
TW\[pq:ss]\[so]\[lr:ns] TB\[aq:cs]\[er:hs]\[ao] C\[Markup at top partially
using compressed point lists (for markup on white stones); listed clockwise,
starting at upper left:\ \- TR (triangle)\ \- CR (circle)\ \- SQ (square)\ \-
SL (selected points)\ \- MA ('X')\ \ Markup at bottom: black & white territory
(using compressed point lists)\ ;LB\[dc:1]\[fc:2]\[nc:3]\[pc:4]\[dj:a]\[fj:b]\[nj:c]\
[pj:d]\[gs:ABCDEFGH]\[gr:ABCDEFG]\[gq:ABCDEF]\[gp:ABCDE]\[go:ABCD]\[gn:ABC]\[gm:AB]\
[mm:12]\[mn:123]\[mo:1234]\[mp:12345]\[mq:123456]\[mr:1234567]\[ms:12345678]\
C\[Label (LB property)\ \ Top: 8 single char labels (1-4, a-d)\ \ Bottom: Labels
up to 8 char length.\ ;DD\[kq:os]\[dq:hs] AR\[aa:sc]\[sa:ac]\[aa:sa]\[aa:ac]\[cd:cj]\
```

\[gd:md\\\[fh:ij\\\[kj:nh\\ LN\[pj:pd\\\[nf:ff\\\[ih:fj\\\[kh:nj\\ C\[Arrows,
 lines and dimmed points.\\) (;B\[qd\\N\[Style & text type\\ C\[There are hard
 linebreaks & soft linebreaks.\\ Soft linebreaks are linebreaks preceded by '\\\\'
 like this one >o\\ k<. Hard line breaks are all other linebreaks.\\ Soft linebreaks
 are converted to >nothing<, i.e. removed.\\ \\ Note that linebreaks are coded
 differently on different systems.\\ \\ Examples (>ok< shouldn't be split):\\ \\
 linebreak 1 "\\n": >o\\ k<\\ linebreak 2 "\\n\\r": >o\\ \\ k<\\ linebreak 3
 "\\r\\n": >o\\ k<\\ linebreak 4 "\\r": >o\\ k<\\) (;W\[dd\\N\[W d16\\C\[Variation
 C is better.\\) (;B\[pp\\N\[B q4\\) (;B\[dp\\N\[B d4\\) (;B\[pq\\N\[B q3\\) (;B\[oq\\N\[B
 p3\\)) (;W\[dp\\N\[W d4\\) (;W\[pp\\N\[W q4\\) (;W\[cc\\N\[W c17\\) (;W\[cq\\N\[W
 c3\\) (;W\[qq\\N\[W r3\\)) (;B\[qr\\N\[Time limits, captures & move numbers\\
 BL\[120.0\\C\[Black time left: 120 sec\\;W\[rr\\ WL\[300\\C\[White time left: 300
 sec\\;B\[rq\\ BL\[105.6\\OB\[10\\C\[Black time left: 105.6 sec\\ Black stones left
 (in this byo-yomi period): 10\\;W\[qq\\ WL\[200\\OW\[2\\C\[White time left: 200
 sec\\ White stones left: 2\\;B\[sr\\ BL\[87.00\\OB\[9\\C\[Black time left: 87 sec\\
 Black stones left: 9\\;W\[qs\\ WL\[13.20\\OW\[1\\C\[White time left: 13.2 sec\\
 White stones left: 1\\;B\[rs\\ C\[One white stone at s2 captured\\;W\[ps\\;B\[pr\\;W\[or\\
 MN\[2\\C\[Set move number to 2\\;B\[os\\ C\[Two white stones captured\\ (at q1
 & r1)\\ ;MN\[112\\W\[pq\\C\[Set move number to 112\\;B\[sq\\;W\[rp\\;B\[ps\\
 ;W\[ns\\;B\[ss\\;W\[nr\\ ;B\[rr\\;W\[sp\\;B\[qs\\C\[Suicide move\\ (all B stones
 get captured)\\)) (;FF\[4\\AP\[Primiview:3.1\\GM\[1\\SZ\[19\\C\[Gametree 2:
 game-info\\ \\ Game-info properties are usually stored in the root node.\\ If games
 are merged into a single game-tree, they are stored in the node\\ \\ where the game
 first becomes distinguishable from all other games in\\ \\ the tree.\\) ;B\[pd\\
 (;PW\[W. Hite\\WR\[6d\\RO\[2\\RE\[W+3.5\\ PB\[B. Lack\\BR\[5d\\PC\[London\\EV\[Go
 Congress\\W\[dp\\ C\[Game-info:\\ Black: B. Lack, 5d\\ White: W. Hite, 6d\\ Place:
 London\\ Event: Go Congress\\ Round: 2\\ Result: White wins by 3.5\\) (;PW\[T.
 Suji\\WR\[7d\\RO\[1\\RE\[W+Resign\\ PB\[B. Lack\\BR\[5d\\PC\[London\\EV\[Go
 Congress\\W\[cp\\ C\[Game-info:\\ Black: B. Lack, 5d\\ White: T. Suji, 7d\\ Place:
 London\\ Event: Go Congress\\ Round: 1\\ Result: White wins by resignation\\) (;W\[ep\\;B\[pp\\
 (;PW\[S. Abaki\\WR\[1d\\RO\[3\\RE\[B+63.5\\ PB\[B. Lack\\BR\[5d\\PC\[London\\EV\[Go
 Congress\\W\[ed\\ C\[Game-info:\\ Black: B. Lack, 5d\\ White: S. Abaki, 1d\\ Place:
 London\\ Event: Go Congress\\ Round: 3\\ Result: Balck wins by 63.5\\) (;PW\[A.
 Tari\\WR\[12k\\KM\\[-59.5\\RO\[4\\RE\[B+R\\ PB\[B. Lack\\BR\[5d\\PC\[London\\EV\[Go
 Congress\\W\[cd\\ C\[Game-info:\\ Black: B. Lack, 5d\\ White: A. Tari, 12k\\ Place:
 London\\ Event: Go Congress\\ Round: 4\\ Komis: -59.5 points\\ Result: Black wins by
 resignation\\)))

sgf (;FF\[4\\GM\[1\\SZ\[19\\FG\[257:Figure 1\\PM\[1\\ PB\[Takemiya Masaki\\BR\[9
 dan\\PW\[Cho Chikun\\ WR\[9 dan\\RE\[W+Resign\\KM\[5.5\\TM\[28800\\DT\[1996-10-18,19\\
 EV\[21st Meijin\\RO\[2 (final)\\SO\[Go World #78\\US\[Arno Hollosi\\ ;B\[pd\\;W\[dp\\;B\[pp\\;
 (;B\[qf\\;W\[rf\\;B\[rg\\;W\[re\\;B\[qg\\;W\[pb\\;B\[ob\\;W\[qb\\) (;B\[mp\\;W\[fq\\;B\[ci\\;W\\
 ;B\[ei\\;W\[eg\\;B\[kk\\LB\[qq:a\\\[dj:b\\\[ck:c\\\[qp:d\\N\[Figure 1\\ ;W\[me\\FG\[257:Figure
 2\\;B\[kf\\;W\[ke\\;B\[lf\\;W\[jf\\;B\[jg\\) (;W\[mf\\;B\[if\\;W\[je\\;B\[ig\\;W\[mg\\;B\[mj\\;
 (;B\[lr\\;W\[qq\\;B\[pq\\;W\[pr\\;B\[rq\\;W\[rr\\;B\[rp\\;W\[oq\\;B\[mr\\;W\[oo\\;B\[mn\\
 (;W\[nr\\;B\[qp\\LB\[kd:a\\\[kh:b\\N\[Figure 2\\ ;W\[pk\\FG\[257:Figure 3\\;B\[pm\\;W\[oj\\;B\\
 ;B\[pi\\;W\[pl\\;B\[qm\\;W\[ns\\;B\[sr\\;W\[om\\;B\[op\\;W\[qi\\;B\[oi\\) (;W\[rl\\;B\[qh\\;W\[

;B\[oh\];W\[np\];B\[no\];W\[mm\];B\[nn\];W\[lp\];B\[kp\];W\[lo\];B\[ln\];W\[ko\];B\[mo\]
;W\[jo\];B\[km\]N\[Figure 3\]) (;W\[ql\]VW\[ja:ss\]FG\[257:Dia. 6\]MN\[1\];B\[rm\];W\[ph\];B\[
;B\[qh\];W\[qe\];B\[sh\];W\[of\];B\[sj\]TR\[oe\]\[pd\]\[pc\]\[ob\]LB\[pe:a\]\[sg:b\]\[si:c\
N\[Diagram 6\])) (;W\[no\]VW\[jj:ss\]FG\[257:Dia. 5\]MN\[1\];B\[pn\]N\[Diagram
5\])) (;B\[pr\]FG\[257:Dia. 4\]MN\[1\];W\[kq\];B\[lp\];W\[lr\];B\[jq\];W\[jr\];B\[kp\];W\[kr\
;W\[hr\]LB\[is:a\]\[js:b\]\[or:c\]N\[Diagram 4\])) (;W\[if\]FG\[257:Dia. 3\]MN\[1\];B\[mf\];W\
3\])) (;W\[oc\]VW\[aa:sk\]FG\[257:Dia. 2\]MN\[1\];B\[md\];W\[mc\];B\[ld\]N\[Diagram
2\])) (;B\[qe\]VW\[aa:sj\]FG\[257:Dia. 1\]MN\[1\];W\[re\];B\[qf\];W\[rf\];B\[qg\];W\[pb\];B\[ol\
;W\[qb\]LB\[rg:a\]N\[Diagram 1\]))

sgf (;FF\[4\]GM\[1\]SZ\[19\]FG\[257:Figure 1\]PM\[2\] PB\[Cho Chikun\]BR\[9
dan\]PW\[Ryu Shikun\]WR\[9 dan\]RE\[W+2.5\]KM\[5.5\] DT\[1996-08\]EV\[51st Honinbo\]RO\[5
(final)\]SO\[Go World #78\]US\[Arno Hollosi\] ;B\[qd\];W\[dd\];B\[fc\];W\[df\];B\[pp\];W\[dq\
;B\[pf\];W\[ke\];B\[id\];W\[lc\];B\[lb\];W\[kb\];B\[jb\];W\[kd\];B\[ka\];W\[jc\];B\[ic\];W\[kb\
;B\[mc\];W\[qc\]N\[Figure 1\] ;B\[pd\]FG\[257:Figure 2\];W\[pc\];B\[od\];W\[oc\];B\[kc\];W\[nd\
(;B\[rf\];W\[md\];B\[kc\];W\[qe\];B\[re\];W\[kb\];B\[mb\];W\[qf\];B\[qg\];W\[pg\];B\[qh\];W\[k\
;B\[hb\];W\[nf\];B\[ch\];W\[cj\];B\[eh\];W\[ob\] (;B\[cc\];W\[dc\];B\[db\];W\[bf\];B\[bb\
;W\[bh\]LB\[of:a\]\[mf:b\]\[rc:c\]\[di:d\]\[ja:e\]N\[Figure 2\] ;B\[qp\]FG\[257:Figure
3\];W\[lo\];B\[ej\];W\[oq\] (;B\[np\];W\[mq\];B\[mp\];W\[lp\] (;B\[kq\];W\[nq\];B\[op\];W\[jq\
;B\[is\];W\[ks\];B\[js\];W\[gq\];B\[gr\];W\[fq\];B\[pq\];W\[pr\];B\[ns\];W\[or\];B\[rq\];W\[hq\
;B\[rr\];W\[cl\];B\[cg\];W\[bg\];B\[og\];W\[ng\] (;B\[ci\];W\[bi\];B\[dj\];W\[dk\];B\[mm\];W\[l\
;B\[mi\];W\[li\];B\[lh\];W\[mg\];B\[ek\];W\[el\];B\[ik\]LB\[kr:a\]N\[Figure 3\]
;W\[ki\]FG\[257:Figure 4\];B\[fl\];W\[fk\];B\[gl\];W\[hk\];B\[hl\];W\[hj\];B\[jl\];W\[kk\];B\[l\
;W\[lm\];B\[ll\];W\[jm\];B\[jj\];W\[ji\];B\[kj\];W\[lj\];B\[ij\];W\[hi\];B\[em\];W\[dl\];B\[ii\
;W\[hh\];B\[ih\];W\[hg\];B\[ln\];W\[kn\];B\[lm\];W\[im\];B\[il\];W\[fg\];B\[lk\];W\[ni\];B\[ef\
;W\[eg\];B\[dg\];W\[ff\];B\[oh\];W\[of\];B\[oj\];W\[ph\];B\[oi\];W\[mj\];B\[ee\];W\[fe\];B\[de\
;W\[ed\];B\[ce\];W\[cf\];B\[rb\];W\[rc\];B\[sc\];W\[qb\];B\[sb\];W\[la\];B\[ma\];W\[na\];B\[ja\
;W\[nb\];B\[la\];W\[pa\];B\[be\];W\[fd\];B\[bj\];W\[ck\];B\[ec\];W\[hs\];B\[gs\];W\[fr\];B\[os\
;W\[ps\];B\[ms\];W\[nk\];B\[ok\];W\[kp\];B\[fo\];W\[fs\];B\[qq\];W\[hs\];B\[do\];W\[co\];B\[ig\
;W\[gc\];B\[gb\];W\[jf\];B\[di\];W\[fi\];B\[hf\];W\[gf\];B\[af\];W\[mo\];B\[he\];W\[kr\];B\[qs\
;W\[no\];B\[oo\];W\[nn\];B\[on\];W\[nl\];B\[ol\];W\[gn\];B\[fn\];W\[in\];B\[nj\];W\[mk\];B\[jg\
;W\[kg\];B\[mi\];W\[jh\];B\[ag\];W\[bk\];B\[ah\];W\[aj\];B\[fh\];W\[fj\];B\[gd\];W\[ra\];B\[dp\
;W\[cp\];B\[go\];W\[gm\];B\[fm\];W\[sd\];B\[se\];W\[ho\];B\[hm\];W\[hn\];B\[ep\];W\[eq\];B\[cd\
;W\[ei\];B\[dn\];W\[gp\];B\[pi\];W\[pf\];B\[dm\];W\[cm\];B\[je\];W\[jd\];B\[if\];W\[ie\];B\[ko\
;W\[jo\];B\[je\];W\[kf\];B\[ni\];W\[dh\];B\[ge\];W\[ie\];B\[rg\];W\[je\]N\[Figure
4\]) (;B\[dk\]FG\[257:Dia. 6\]MN\[1\];W\[ck\];B\[gk\]N\[Diagram 6\])) (;B\[nq\]VW\[ai:ss\]FG\[2
5\]MN\[1\];W\[mr\];B\[nr\];W\[lr\]TR\[oq\]N\[Diagram 5\])) (;B\[mp\]VW\[ai:ss\]FG\[257:Dia.
4\]MN\[1\];W\[op\];B\[oo\];W\[no\];B\[mo\];W\[on\];B\[po\] ;W\[mn\];B\[np\];W\[nn\];B\[or\]N\[D
4\])) (;B\[rc\]VW\[aa:sj\]FG\[257:Dia. 2\]MN\[1\];W\[rb\];B\[sb\];W\[la\];B\[ma\];W\[na\];B\[j
;W\[pa\]N\[Diagram 2\]) (;B\[rb\]VW\[aa:sj\]FG\[257:Dia. 3\]MN\[1\];W\[rc\];B\[sc\];W\[qb\];B\
;W\[sd\];B\[qa\]N\[Diagram 3\])) (;B\[qf\]VW\[aa:sj\]FG\[257:Dia. 1\]MN\[1\];W\[mb\];B\[kc\];W\
;W\[la\];B\[nb\];W\[eb\]LB\[ob:a\]\[na:b\]\[rc:c\]\[sd:d\]N\[Diagram 1\]))
